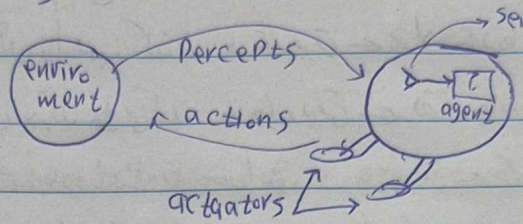
هوش مصنوعی

DATE : / / SUB :

Amin

لکچر 2 :

Agent : هر چیزی است که بتوان آن را به عنوان یک موجودیت در نظر گرفت که محیط خود را از طریق حسگر ها (sensors) درک می کند و از طریق عملگر ها (actuators) بر آن تاثیری گذارد.



(دنباله ادراکات)

Percept sequence : دنباله ادراکات مجموعه ای از تمام ورودی هایی است که یک عامل از ابتدا تا لحظه ی خاص دریافت کرده است، این دنباله مبنای تصمیم گیری عامل است. ★ عامل نمی تواند بر اساس چیزی که ندیده یا تجربه نکرده است، تصمیم بگیرد، بلکه فقط از اطلاعاتی که تاکنون دریافت کرده استفاده می کند.

Agent function : تابع عامل، هر دنباله ی ادراک ممکن را به یک عمل خاص نگاشت می کند، مشخص می کند که عامل در پاسخ به هر دنباله ی ادراک ممکن، چه اقدامی باید انجام دهد. به بیان دیگر، این تابع تمام رفتارهای عامل را تعیین می کند

مثال جاروبرقی :

percepts: Location and status

Actions: Left, Right, Clean, NoOP

Function Vacuum-Agent → if status = Dirty then return clean
else if location = A then return Right else if location=B the return Left

Rational Agent (عامل منطقی) : یک عامل منطقی همیشه سعی می کند بهترین تصمیم ممکن را بگیرد تا به بالاترین سطح عملکرد دست یابد. این تصمیمات بر اساس دو چیز گرفته می شوند. ① دنباله ادراکات (اطلاعاتی که عامل تاکنون دریافت کرده است) ② دانش درون عامل، قوانینی که از قبل در عامل برنامه ریزی شده اند.

Preformance measure (معیار عملکرد - تابع سودمندی) : این تابع تعیین می کند که یک عامل چقدر خوب عمل کرده است یعنی مشخص می کند کدام تصمیمات بهتر هستند و کدام تصمیمات بدتر.

نکته ★ معیار عملکرد عامل جاروبرقی، میزان گرد و غباری است که در یک شیفت 8 ساعته تمیز می کند. این عامل می تواند با تمیز کردن گرد و غبار، پس گرفتن دوباره آن از روی زمین و دوباره تمیز کردن، این معیار را به حداکثر برساند، برای جلوگیری از چنین تقلبی می توان امتیاز را بر اساس تعداد خانه های تمیز در هر بازه زمانی انجام داد (واقعا برای مصرف برق و ایجاد نویز جریمه در نظر گرفت).

در هوش مصنوعی، محیط وظیفه ی عامل با استفاده از عبارت مخفف PEAS مشخص میشود.

SOBHAN

DATE : / / SUB :

PEAS = ۱) Performance (معیار عملکرد) Measure ۲) Environment (محیط) ۳) Actuators (عمل‌گرها) ۴) Sensors (حسگرها)

P: تابعی که عامل آن را بیشینه یا کمینه می‌کند، این تابع مشخص می‌کند که چه چیزی یک عامل را موفق می‌سازد. این معیار همان چیزی است که عامل سعی می‌کند آن را بیشینه کند، در جاروبرقی ← تعداد خانه‌های تمیز در یک بازه زمانی

E: محیط، تمام چیزهای دنیای واقعی است که عامل در آن عمل می‌کند. در جاروبرقی ← نقشه خانه، مبل‌های کثیف

A: مجموعه‌ای از اعمالی که عامل می‌تواند انجام دهد و باعث تغییر محیط شود. جاروبرقی ← حرکت به جلو یا راست یا...

S: مشاهداتی که به عامل اجازه می‌دهد ماهیت محیط را بررسی کند جاروبرقی ← حسگر کثیفی و تشخیص موانع

مثال PEAS برای یک تاکسی خودران

P ← سفر امن، سریع، قانونی و راحت / E = جاده‌ها، ترافیک، عابران پیاده / A: فرمان، پدال و ترمز

S: دوربین‌ها، کیلومترشمار، GPS، حسگر موتور

انواع محیط

۱) Fully observable vs Partially observable

(عامل کل محیط را می‌بیند) (زاویه دید محدود است)

۲) Deterministk vs Stochastic

(قطعی) وضعیت بعدی کاملاً قابل پیش‌بینی است. (تصادفی) خیر، وضعیت بعدی با احتمال مشخص بین چند حالت ممکن تغییر می‌کند

۳) Episodic vs Sequential

(هر تصمیم به طور مستقل گرفته می‌شود و ترتیب مهم نیست. عامل صرفاً باید در هر اپیزود بهترین تصمیم ممکن را بگیرد) (تصمیم‌های فعلی بر وضعیت بعدی تأثیر می‌گذارند و ترتیب مهم است. عامل باید پیامدهای بلندمدت تصمیم‌هایش را در نظر بگیرد)

۴) static vs dynamic

این دسته‌بندی تعیین می‌کند که آیا محیط در طول زمان تغییر می‌کند یا نه.

محیط ثابت به زمان بستگی ندارد / محیط در حال تغییر به زمان محدود است

۵) Discrete vs continuous

گسسته / پیوسته

۶) single-agent vs multi-agent

نامحدود و پیوسته

۷) Known vs Unknown SOBHAN

آیا عامل، قوانین محیط، پاداش‌ها و مدل‌ها و... را می‌شناسد یا خیر؟

DATE : / / SUB :

لکچر 3

Solving Problems by searching

انواع Agent ها :

(عامل برنامه‌ریز) Planning_Agent — تصمیم بر اساس پیشبرد نتایج آینده، نیاز به مدل پیچیده و زمانبر، نیاز به هدف

(عامل واکنشی) Reflex-Agent — تصمیم بر اساس ادراک فعلی، مدل ساده و سریع، نیاز به هدف ندارد، پیچیدگی ساده

★ توی الگوریتم های سرچ، عامل ها در محیط های کشف شده (قابل رویت همه جا) Fully observable, deterministic, (متعین) شناخته شده هستند

مولفه های مسئله جستجو :

① Initial state ② Actions : لیست اقداماتی که عامل می تواند انجام دهد.

③ Transition model : مشخص می کند که اگر در یک وضعیت خاص، یک اقدام انجام دهیم، چه وضعیتی حاصل میشود.

④ Goal state : مشخص می کند که وقتی برآورده شود، مسئله حل شده است. ⑤ Path cost : هزینه ریاضی از وضعیت اولیه به هدف

Optimal soultion : راه حل بهینه مسیر یا توالی اقداماتی که کمترین هزینه مسیر را برای رسیدن به هدف دارد.

مثالش آسونه توی کلاس بین

State space : فضای حالت مجموعه‌ای از تمام وضعیت هایی است که میتوان از وضعیت اولیه با استفاده از دنباله‌ای اقدامات به آنها رسید. مثال ها بدیهی است توی جزوه هست

خیلی جزئیات و آسونه ← ایده اصلی جستجو :

① عامل از یک نقطه شروع حرکت را آغاز می کند ② تمام وضعیت های جانشین را بررسی می کند ③ یک لیست از وضعیت هایی که هنوز توسعه داده نشده اند نگه می‌داریم ④ یکی از وضعیت های موجود را انتخاب میکنیم و آن را گسترش می دهیم ⑤ اگر وضعیت نهایی، هدف یا راه حل بود تمام ⑥ این فرایند ادامه پیدا می کند تا هدف پیدا شود.

مثل انیمیشن توی کلاس خیلی باحاله مثل 18 تقریبا بلدم، اومده درخت و tree search یادمون

لکچر 4

استراتژی های سرچ غیرآگاهانه Uninformed search strategies

الگوریتم هایی که به ترتیب گسترش گره ها مرتبه هستند و فقط از اطلاعات موجود در تعریف مسئله برای جستجو استفاده می شود و هیچ گونه اطلاعات اضافی در نظر گرفته نمی شود.

انواع آنها :

① Breadth-first search (BFS)
② Depth-first search (DFS)
③ Iterative deepening search (IDS)
④ Uniform_cost search (UCS)

SOBHAN

DATE : / / SUB :

(FIFO)

BFS : اینارو که بلدی اگه چیزی جالب و جدید بود میتونیم : (پیاده سازی با queue بود)

DFS : پیاده سازی با استک بود (LIFO)

معیار های ارزیابی استراتژی های جستجو :

① کامل بودن : استراتژی هایی مانند BFS معمولاً کامل هستند، زیرا تمام گره ها را بررسی می کنند تا زمانی که هدف پیدا شود.

② بهینگی (optimality) : UCS معمولاً بهینه است زیرا گره ها را بر اساس کمترین هزینه گسترش می دهد. اما dfs ممکن است جواب بهینه پیدا نکند.

③ پیچیدگی زمانی : (بعدا خودمون اورد خودمون) که واسه BFS و DFS هست $O(n+m)$ n^2

④ پیچیدگی فضایی : معمولاً حافظه مورد نیاز برای BFS، پیچیدگی فضایی میتواند به $O(n^2)$ برسد، در مقابل dfs معمولاً پیچیدگی فضایی کمتری دارد، زیرا فقط مسیر فعلی را ذخیره می کند.

نکته : اگر فضای جستجو بزرگ باشد dfs خوبه که حافظه کمتری نیاز دارد، و اگر مسیر بهینه مهم باشد، باید از UCS یا BFS استفاده کرد.

ویژگی های BFS : کامل ✓ بهینه ✓ تایم $O(b^d)$ حافظه $O(b^d)$

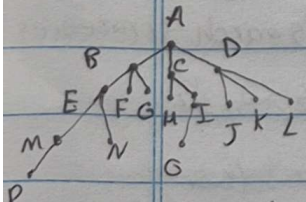
ویژگی های DFS : کامل X (توی حلقه گیر نکنه) بهینه X تایم : $O(b^m)$ حافظه $O(bm)$

★ اگه dfs بدون واپس گرد های تکراری نشه :

کامل ✓ بهینه ✓ تایم = $(d+1)b^0 + db^1 + (d-1)b^2 + \dots + b^d$ فضا $O(bd)$

IDS :

ترکیبی از dfs و bfs هست. ابتدا عمق محدودی رو انتخاب می کنه بعد تا اون عمق dfs میزنه و اگه پیدا نشد عمق رو یکی افزایش میدیم و از اول شروع می کنیم اینجوری مثل bfs لازم نیست که حافظه زیادی را ذخیره کنیم و مثل dfs سریعتره مثال :



bfs: A, B, C, D, E, F, G, M, I, J, K, L, M, N, O, P

dfs: A, B, E, M, P, N, F, G, C, H, I, O, D, J, K, L

Ids → ۰ عمق → A / ۱ عمق → A, B, C, D / ۲ عمق → A, B, E, F, G, C, H, I, D, J, K, L

۳ عمق → A, B, E, M, N, F, G, C, H, I, O, J, K, L

۴ عمق → A, B, E, M, P ← dfs مثل

SOBHAN

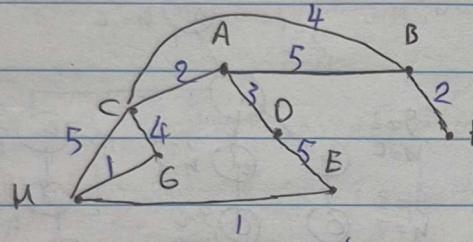
DATE : / / SUB :

ویژگی های IDS ← کامل ✓ ، بهینه هست ✓ ، تابع $(d+1)b^0 + db^1 + (d-1)b^2 + \dots + b^d$ ، حافظه $O(bd)$

UCS :

هدف این است که کمترین هزینه را برای رسیدن به هدف پیدا کنیم به این معنی که در UCS در هرگام از جستجو، گره ای که کمترین هزینه از نقطه شروع به آن گره دارد را گسترش می دهیم.
برای هر گره مرزی ، هزینه مسیر از مرحله اول تا آن گره را ذخیره می کنیم و اون گره ای که هزینه کمتری دارد را گسترش می دهیم برای پیاده سازی هم با صف اولویت دار درست می کنیم اگر همه هزینه ها برابر باشد این الگوریتم همان bfs است.

مثال خیلی خوبه :



① گره های مرزی ← A با هزینه 0

② گسترش A ← گره های مرزی $(B(5), D(3), C(2))$

③ گسترش C ← گره های مرزی $(B(5), D(3), G(2+4), H(2+5))$

④ گسترش D ← گره های مرزی $(B(5), G(2+4), H(2+5), E(3+5))$

⑤ گسترش B ← گره های مرزی $(G(6), H(7), E(8), F(5+2))$

⑥ گسترش G ← گره های مرزی $(H(7), E(8), F(7))$

⑦ گسترش H و تمام $A \to C \to G \to H$

ویژگی ها UCS : کامل ✓ ، بهینه ✓

تابع [illegible] ، فقط بستگی به مسیرمون داره. فرق نکنه 4 جدول خلاصه داره نکته ③ املی

سرچ های آگاهانه : ① Heuristics ② Greedy search ③ A* search

خلاصه (UCS) : استراتژی : گسترش دادن کمترین هزینه مسیر

خوبی هاش : کامل و بهینه است بدی ها : آپشن ها رو در همه جهت ها می گرده و اطلاعاتی درباره مکان هدف نداره.

Heuristic یک تابع تخمین است که مقدار نزدیکی یک وضعیت به هدف را مشخص می کند و برای مسائل خاص طراحی میشود و در جستجو های آگاهانه برای هدایت مسیر جستجو به سمت هدف استفاده می شوند. به الگوریتم کمک می کند تا مسیر های بهینه تر را سریع تر پیدا کند به طور خلاصه به الگوریتم کمک می کند تا بدون بررسی تمام حالات، سریع تر به جواب برسد.

Greedy search تو جستجو حریصانه ، همیشه گره ای که به نظرش نزدیک ترین مسیر به هدف را دارد گسترش می دهیم.

SOBHAN

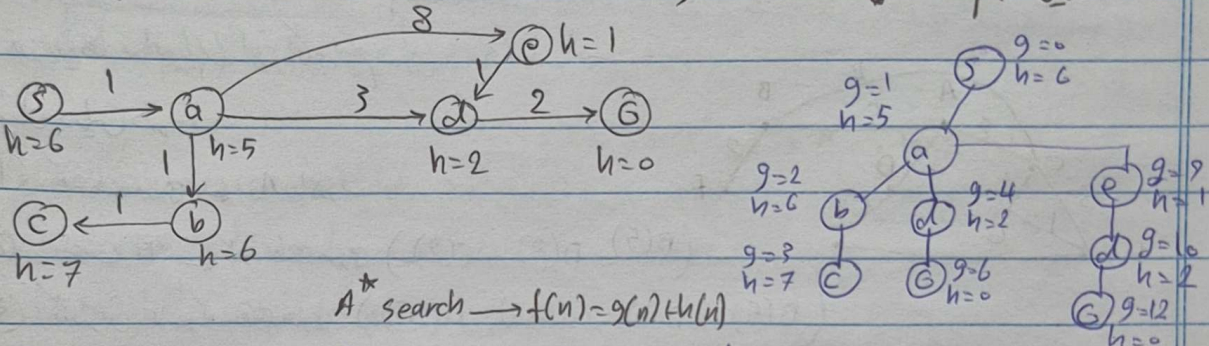
توی هیورستیک تابعی که تخمین می زنه یک وضعیت چقدر به هدف نزدیک است.

DATE : / / SUB :

مشکلات جستجوی حریصانه: ① به سمت مسیر اشتباه کشیده می‌شود: اگر هیوریستیک دقیق نباشد ممکن است الگوریتم بدون در نظر گرفتن هزینه مسیر واقعی، به سمت یک مسیر غلط برود. مثلاً ممکن است سریع‌ترین مسیر ظاهری را انتخاب کند ولی بعد متوجه شود که یک مانع بزرگ در مسیر وجود دارد.

② در بعضی موارد، مثل DFS عمل می‌کند: ممکن است به کوه‌های بروند که به نقاط تردیک هستند ولی در واقع بن‌بست‌اند. در بدترین حالت، مثل DFS غیرهوشمندانه رفتار می‌کند و راه بهتر را پیدا نمی‌کند

مثال خیلی مهم ← UCS → g(n), Greedy → h(n) هدف ← رسیدن از S به G



نوع UCS براساس هزینه طی شده از نقطه شروع g(n) کار می‌کند و مسیر کم هزینه‌تر را انتخاب می‌کند نوع greedy فقط به هزینه تخمینی تا هدف h(n) نگاه می‌کند نزدیک‌ترین گره از نظر فاصله را انتخاب می‌کند و ممکن است به یک مسیر غلط کشیده شود.

چرا A^* بهتر است؟ برخلاف گریدی که فقط به تخمین h(n) نگاه می‌کند مقدار هزینه واقعی طی شده را هم در نظر میگیرد و برخلاف UCS که فقط g(n) را در نظر می‌گیرد A^* می‌تواند مسیرهای کوتاه‌تر و بهتر را پیدا کند. پس A^* بهینه، کامل و کارآمد است به شرطی که تابع هیوریستیک خوب طراحی شده باشد.

آیا A همیشه بهینه است؟

بله اما به شرطی که تخمین هیوریستیک درست باشد. مشکل وقتی پیش می‌آید که تخمین بیش از حد خوشبینانه نباشد یعنی مسیر بد واقعی هزینه‌اش کمتر از مسیر خوبی که تخمین زده شده کد یعنی یک مسیر خوب را نادیده می‌گیریم.

راه حل: باید ویژگی Admissible را داشته باشد.

Admissible: مقدار h(n) نباید بیشتر از هزینه واقعی رسیدن به هدف باشد، یعنی همیشه باید خوشبینانه باشد

بهینگی A^*: فرض کنیم A یک گره بهینه است، B یک گره غیربهینه است. و Admissible h(n) است.

ادعا: گره A قبل از گره B از صف Fringe (مرز جستجو) خارج خواهد شد.

Proof: فرض کنیم B روی مرز قرار دارد (یعنی هنوز بررسی نشده است). همچنین فرض کنیم یک گره اجدادی n از مسیر A روی مرز وجود دارد (ممکن است خود A باشد) چون تابع A^* مقدار f(n) را به این صورت محاسبه می‌کند: $f(n) = g(n) + h(n)$ / g(n): هزینه رسیدن به n از آنجا که A یک مسیر بهینه دارد،

h(n): برآورد هزینه باقی‌مانده تا هدف

داریم: $f(n) \leq f(A)$ و $f(A) < f(B)$ بنابراین مقدار f(n) از f(B) کمتر است، به این معنا که A قبل از B گسترش می‌یابد

SOBHAN

DATE : / / SUB :

از GPT پرسیدم h(n) دو با چه فرمولی بدست می آورم؟ منیدم که در ادامه توضیح خود اسلایدها توضیح میدن ولی خب یه مثال هم داد:

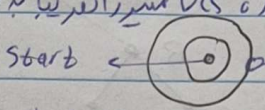
در مسئله ای که باید مسیر بهینه از یک نقطه به نقطه دیگر را بیابیم (مثلاً در نقشه حرکتی یا بازی های کامپیوتری) می توانیم از فاصله تخمینی بین دو نقطه به عنوان h(n) استفاده کنیم. اگر حرکت فقط به صورت افقی و عمودی مجاز باشد، (مثلاً در یک شهری) می توان از این فرمول استفاده کرد: $h(n) = |x_{Goal} - x_n| + |y_{Goal} - y_n|$

مثلاً یا اگر حرکت در همه جهات مجاز باشد فاصله مستقیم را استفاده می کنیم:

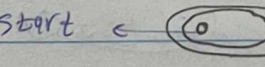
$$h(n) = \sqrt{(x_n - x_{Goal})^2 + (y_n - y_{Goal})^2}$$

تفاوت UCS و A^* در نحوه گسترش گره ها:

UCS همیشه گره هایی را که کمترین هزینه تجمعی $g(n)$ دارند گسترش می دهد. این الگوریتم در همه جهات ممکن گسترش پیدا می کند زیرا هیچ تخمینی از فاصله تا هدف ندارد و فقط روی هزینه مسیر طی شده تمرکز دارد. اگر همه هزینه ها برابر باشند، UCS شبیه به BFS عمل می کند. مثال: اگر نقشه ای داشته باشیم که در آن هزینه حرکت در همه جهات مساوی باشد، UCS مسیر را تقریباً به صورت دایره ای گسترش می دهد تا به هدف برسد.



A^* هم هزینه مسیر $g(n)$ را در نظر میگیرد و هم از یک تابع هدایتی $h(n)$ برای تخمین هزینه باقیمانده استفاده می کند، این باعث میشود که A^* برخلاف UCS، بیشتر به سمت هدف حرکت کند با این حال اگر مسیر های جانبی ای وجود داشته باشد که ممکن است بهینه تر باشند، A^* آن ها را هم تا حدی بررسی می کند در یک نقشه A^* به جای اینکه همه جهات را به طور یکنواخت جستجو کند، بیشتر در جهتی حرکت می کند که نزدیکترین مسیر به هدف است، اما اگر مسیرهای دیگر نیز امیدوار کننده باشند، آن ها را هم تا حدی بررسی می کند.

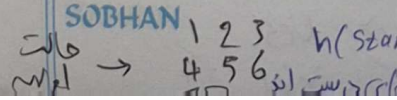


کاربرد های A^* در دنیای واقعی: ① بازی های ویدئویی ② حل مسائل مسیریابی و ... سوپر توضیح:

پیاده کردن تابع Heuristic برای مسئله 8-Puzzle تعریف مسئله: وضعیت ها: هر چینش ممکن از 8 عدد و یک مقدار خالی در مربع 3×3 / تعداد وضعیت ها = $\frac{9!}{2}$ / Action: بالا، پایین، چپ، راست / تعداد جانشین ها: بسته به موقعیت مقدار خالی بین 2 تا 4 جانشین / cost = معمولاً 1 واحد برای هر حرکت

① Heuristic: تعداد کاشی های جابجا شده: این تابع تعداد کاشی هایی را میشمارد که در مکان درست خود نیستند. چرا این تابع Admissible است؟ چون مقدار این تابع همیشه کمتر یا برابر هزینه واقعی رسیدن به جواب است. زیرا هر کاشی که در مکان نادرست است، حداقل یک جابجایی برای رسیدن به موقعیت درست خود نیاز دارد.

SOBHAN



A* بهینه است اگر هم admissible و هم consistency باشد

DATE : / / SUB :

Heuristic2 مجموع فاصله های منهتن: این تابع مجموع فاصله های هر کاشی تا مکان درست خود را میشمارد
مجدول: $h(n) = \sum_{i=1}^{8} |x_i - x_{Goal}| + |y_i - y_{Goal}|$ / چرا Admissible است؟ چون حرکت ها فقط به سمت
آفقی و عمودی است، بنابراین این تابع حداقل تعداد حرکات لازم برای رسیدن به جواب را تخمین می زند.

مثال: وضعیت اولیه → 8 1 3 / 4 _ 2 / 7 6 5 — هدف → 1 2 3 / 4 5 6 / 7 8 _ — جواب = 2+0+3+1+2+1+2+2 = 6

سوال: آیا استفاده از هر تابع دلخواه به عنوان heuristic درست است؟ آن وقت این تابع
کاملاً دقیق است ولی باعث میشود A* تمام مسیرهای ممکن را محاسبه کند یعنی دیگر مزیتی ندارد!
در واقع این حالت مثل اجرای UCS است که بهینه است ولی بهره ای از heuristic نمی برد.
همچنین میتونیم چندتا h را با هم ترکیب کنیم و بهترین استفاده رو ازشون کنیم.

Graph search

میرات گراف رو تبدیل به درخت کنیم تا راس های تکراری را شناسایی نکنیم:

گراف → A, B, C, D — درخت → A, B, B, C, A, A, C

منظورمان اینه که راس های تکراری رو expand نکنیم و توی لیست بیایم مثلا به جای
لیست از set استفاده کنیم برای تکراری، راس های مرزی تا راس تکراری را اکسپند نکنیم

Consistency (سازگاری): به این معناست که اگر ما از گره A به گره C میرویم
$h(A) - h(C) \leq cost(A \to C)$ / یعنی هیچ گاه از A به C هزینه کمتر نباید باشد از این رابطه زیر برقرار باشد
$h(A) \leq cost(A \to C) + h(C)$
آن موقع A* گراف سرچ بهینه میشود.
A* با استفاده از h(n) سازگار همیشه بهینه است و قادر است بهترین مسیر را پیدا کند
برای اثبات اولیت اینکه Admissible یا Consistency؟ سازگاری (consistency)

A* graph search is optimal

A* : Tree search بهینه است اگر h(n) admissible باشد، UCS یک حالت خاص است اگر h=0
A* : Graph search بهینه است اگر h(n) consistency باشد، UCS بهینه است چون h=0

نتیجه گیری: A* در حیث رجو درختی بهینه است اگر h(n) آن Admisible باشد
A* در حیث رجو گرافی بهینه است اگر h(n) آن consistency باشد.

SOBHAN

UCS در هر دو حالت بهینه است اگر h=0
اگر h(n) سازگار باشد آنگاه مجاز هم است.

DATE : / / SUB :

Artificial Inteligence (هوش مصنوعی)
local search (جستجو محلی)

Hill climbing یک الگوریتم جستجوی محلی ساده است که برای پیدا کردن بهترین جواب (یا لزوماً بهترین جواب) استفاده می شود. ایده کلی این الگوریتم به این صورت است:

① شروع از هر جایی: از یک وضعیت ابتدایی شروع می کند

② حرکت به وضعیت بهترین همسایه: در هر مرحله، از وضعیت فعلی به همسایه‌ای که بهترین حالت را دارد (بر اساس معیار مورد نظر) حرکت می کند.

③ اتمام جستجو در صورت عدم بهبود: اگر هیچ همسایه ای بهتر از وضعیت فعلی پیدا نشود، جستجو تمام میشه.

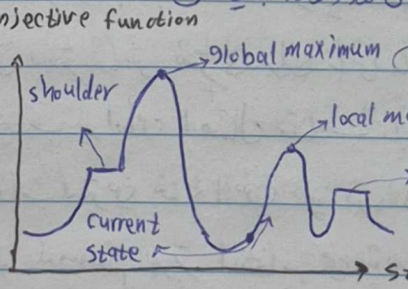
چرا از این الگوریتم استفاده می کنیم؟ چون توی خیلی از مسائل فضای جستجو بسیار بزرگ است و جستجو در تمام آن فضا عملی نیست و همیشه همه حالت های ممکن را نمیشه با bfs و dfs و ... جواب داد. دلیل دیگر: در بسیاری از مسائل، هدف پیدا کردن وضعیت هدف است، نه مسیری که به آن وضعیت منتهی می شود. به عنوان مثال: در مسئله n-queens هدف پیدا کردن یک پیکربندی از خانه هاست که تمام محدودیت ها را رعایت کند.

جمع بندی: الگوریتم های ساده جستجو مانند Hill climbing برای مسائل بهینه سازی مناسب هستند، زیرا آن ها نیاز به جستجو در فضای بزرگ ندارند و می توانند به سرعت به سمت بهینه ترین وضعیت حرکت کنند بدون اینکه مسیر دقیقاً مهم باشد.

معایب: معمولاً کامل نیست و ممکن توی حلقه گیر کند و همچنین optimal هم نیستند.

مزایا: حافظه کمی مصرف می کند و قادر به یافتن جواب های معقول در فضای جستجوی بزرگ یا بی نهایت هستند.

Hill climbing Diagram



★★ ایده اصلی (شبیه سازی آنیلینگ): اجازه داده می شود که حرکت های بد (حرکت هایی که وضعیت بهینه را بدتر می کنند) به طور تصادفی انجام شوند. اما میزان این حرکت ها با گذشت زمان کاهش می یابد. در ابتدا حرکت های بد باعث می شوند که سیستم از نقاط محلی (مثل max و min های محلی) خارج شود و به دنبال بهترین جواب جهانی بگردد. با گذشت زمان احتمال انجام حرکت های بد کمتر می شود و این باعث میشود که سیستم در نهایت به یک جواب خوب یا بهینه نزدیک تر شود.

SOBHAN

DATE : / / SUB :

مراحل الگوریتم: ① ابتدا یک وضعیت تصادفی را به عنوان وضعیت اولیه انتخاب می کنیم.
② یک حرکت تصادفی از وضعیت فعلی به وضعیت جدید پیشنهاد می دهیم.
③ اگر وضعیت جدید بهتر از وضعیت فعلی باشد به آن وضعیت می رویم.
④ اگر وضعیت جدید بدتر از وضعیت فعلی باشد، با احتمال معینی این حرکت بد را می پذیریم تا سیستم از نقاط محلی خارج شود. احتمال $\leftarrow e^{-\frac{\Delta E}{T}}$ تابع نمایی است ($\Delta E \leftarrow$ تفاوت اختلاف ارتفاع)
⑤ این روند را تکرار می کنیم و احتمال اینکه از نقاط محلی خارج شویم را کاهش می دهیم.

Genetic Algorithm

یک روش بهینه سازی تکاملی است و مجموعه ای از راه حل های ممکن (جمعیت) به مرور زمان تکامل پیدا می کنند تا به بهترین جواب ممکن برسند.

مفهوم کلی: این الگوریتم شبیه به فرایند انتخاب طبیعی در زیست شناسی است:
در طبیعت موجوداتی که قوی تر هستند، شانس بیشتری برای زنده ماندن و تولید مثل دارند.
در الگوریتم ژنتیک: راه حل هایی که بهتر عمل می کنند، احتمال بیتری برای انتخاب و ترکیب شدن با یکدیگر دارند، در حالی که راه حل های ضعیف تر حذف می شوند.

مراحل الگوریتم ژنتیک:
① ایجاد جمعیت اولیه: یک مجموعه از راه حل های تصادفی (که به آنها کروموزوم یا فرد گفته می شود) تولید میشود. هر کروموزوم یک کاندید برای حل مسئله است.
② ارزیابی (Evalution - fitness function): هر کروموزوم بر اساس یک تابع ارزیابی (fitness function) سنجیده می شود تا مشخص شود که چقدر به جواب مطلوب نزدیک است.
③ انتخاب (selection): راه حل بهتر شانس بیتری برای باقی ماندن دارند، برخی روش های انتخاب شامل:
• Roulette wheel selection: انتخاب بر اساس احتمال متناسب با مقدار fitness.
• Tournoment selection: انتخاب بهترین فرد از بین چند فرد تصادفی.
④ ترکیب (cross over - mating): دو کروموزوم برتر انتخاب می شوند و ترکیب شده (مقداری از اطلاعات آنها با هم ترکیب می شود) تا فرزندان جدید تولید شوند.
⑤ جهش (mutation): برخی از کروموزوم ها با احتمال کمی جهش می یابند (مثلاً تغییر یک مقدار تصادفی در بخشی از کروموزوم) تا تنوع در جمعیت حفظ شود و از گیر افتادن در ماکسیماهای محلی جلوگیری شود.
⑥ تکرار مراحل 2 تا 5: این روند ادامه میابد تا جمعیت تکامل یابد و به راه حلی بهینه برسیم یا یک معیار توقف برقرار شود (مثلاً بعد از تعداد مشخصی از نسل ها)

SOBHAN

DATE : / / SUB :

روش های رایج crossover (ترکیب شدن)

- one-point crossover : نقطه ای تصادفی در کروموزوم انتخاب شده و تغییر اطلاعات بین والدین صادر می شود.
- Two-Point crossover : دو نقطه تصادفی مشخص شده و بخش میان مبادله می شود.
- Uniform crossover : هر بیت از کروموزوم میتواند با احتمال 50% از یکی از والدین کپی شود.

مثال از اولی ← والدین : $P_1: 101110$ $P_2: 110101$ بعد از crossover در نقطه 3 : $O_1: 101101$ $O_2: 110110$

بعد از ترکیب والدین، هر فرزند با احتمال Pm (معمولاً 0.01 یا 0.05) دچار جهش می شود.

مثال: قبل از جهش: 101110 بعد از جهش (با احتمال تغییر 5% برای هر بیت): 100110

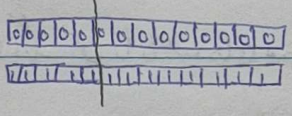
خلاصه فرایند چرخه تولید مثل در GA :

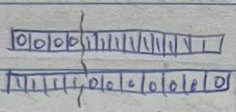
① انتخاب والدین (افرادی که شانس تولید مثل دارند). ② تصادفی سازی ترتیب والدین در جفت گیری

③ ترکیب والدین (crossover) برای تولید نسل جدید ④ اعمال جهش (mutation) روی فرزندان.

⑤ حذف نسل قبلی و جایگزینی با نسل جدید. / این فرآیند در هر نسل تکرار میشود تا به جواب بهینه برسیم.

توضیحات کاملتر از روش های کراس اور :

1-Point crossover Parents 

children 

n-Point crossover Parents: AA|AAA|A|AA / BB|BBB|B|BB children: AA|BBB|A|BB / BB|AAA|B|AA

uniform crossover این همش رندومه ارتقا سکه انداختیم اگه رو آمد تغییر میکنه اگه نه نکنه

مزایا ژنتیک: سرعت تغییر و نیاز به حافظه کمتر / ساده بودن

معایب ژنتیک: رندوم بودن - بهینه و کامل نیست / امکان گیر افتادن در نقاط در اکثر محلی / سختی در انتخاب بهترین نمایندگان برای کاندیدها

Artificial Intelligence: Constraint Satisfaction Problem (CSP) (مسائل رضایت از محدودیت ها)

یک کلاس از مسائل در هوش مصنوعی هستند که در آنها هدف پیدا کردن مقدارهایی برای متغیرهاست، به طوری که تمام محدودیت ها رعایت شوند.

اجزای اصلی یک CSP: ① مجموعه ای از متغیرها مثل $x_1, x_2, \ldots, x_n$ ② دامنه ای از مقادیر ممکن برای هر متغیر ③ مجموعه ای از محدودیت ها که تعیین می کند کدام ترکیب از مقدارها معتبر است.

DATE : / / SUB :

مثال‌های معروف CSP: ① مسئله رنگ آمیزی گراف ② مسئله N-Queen ③ سودوکو

روش‌های حل CSP: ① backtracking ② forward checking: بررسی تاثیر هر مقدار روی متغیر های آینده برای کاهش فضای جستجو ③ Arc consistency: حذف مقدار های ناسازگار از دامنه متغیر ها برای سرعت بخشیدن به حل

دو تا از انواع مسائل سرچ:

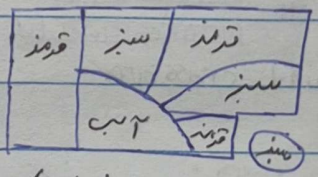
① جستجو برای برنامه ریزی (Search for Planning): در این نوع جستجو، مسیر به سمت هدف مهم است یعنی خود مسیر و چگونگی رسیدن به هدف اهمیت دارد نه فقط هدف نهایی. مثل UCS و DFS, BFS و ... A*

② جستجو برای تخصیص (Search for Assignment): در این نوع جستجو، هدف پیدا کردن یک تخصیص کامل و سازگار از مقدار ها برای متغیر ها است. مسیر رسیدن به راه حل مهم نیست بلکه فقط خود تخصیص نهایی اهمیت دارد. مثال: حل سودوکو، رنگ آمیزی گراف و ... که با CSP حل میشوند.

شرایط یک حوزه معتبر برای CSP: ① کامل بودن (complete Assignment): تمام متغیر ها مقدار گرفته باشند ② سازگاری (consistent Assignment): هیچ محدودیتی نقض نشده باشد.

مزایای CSP نسبت به جستجوی درختی معمولی: ① کاهش فضای جستجو ② استفاده از روش های تخصیص تر

مثال: این ناحیه ها رو با قرمز، سبز و آبی رنگ کن تا ناحیه های مجاور، هم رنگ نباشند.



توضیحات: ادامه در اینجا با یک تراکت بود که چون بلدم ننوشتم.

تشخیص زود هنگام شکست (Early Detection of failure) در CSP:

هدف این تکنیک، کاهش فضای جستجو و تشخیص زود هنگام مسیر های نامعتبر است تا از ادامه مسیر های اشتباه جلوگیری شود. اینکار با استنتاج (Inference) انجام میشود. یکی از معمولترین روش ها برای این کار Forward checking است.

Forward checking (بررسی رو به جلو): ایده اصلی: وقتی یک متغیر مقدار میگیرد، باید بررسی کنیم که چگونه این مقدار روی دامنه متغیر های دیگر تاثیر می گذارد. اگر مقدار انتخاب شده باعث شود که دامنه یک متغیر دیگر خالی شود، یعنی دیگر مقدار مجازی برای آن باقی نمانده و مسیر فعلی به شکست خواهد انجامید. در اینصورت، جستجو را زودتر متوقف کرده و به عقب بازمیگردیم. (Backtrack زود هنگام)

SOBHAN

DATE : / / SUB :

خلاصه Forward checking : وقتی یک متغیر را مقداردهی می‌کنیم، بلافاصله بررسی می‌کند که چه تاثیری بر دامنه‌ی متغیرهای دیگر دارد. مثال ساده: فرض کنیم دو متغیر داریم: X و Y، و محدودیت این که $X \neq Y$. دامنه‌های اولیه، $X=\{1,2,3\}$ ، $Y=\{1,2\}$

مقداردهی به X: اگر $X=1$ باشد، دامنه Y به $\{2\}$ کاهش پیدا می‌کند.

این فقط بعد از مقداردهی به X انجام می‌شود. اگر یک متغیر چند مقدار بگیرد، برای آن متغیر، دامنه‌های دیگر را بررسی می‌کنیم.

خلاصه Arc consistency :

اینجا بررسی انجام می‌شود که آیا برای هر مقدار از یک متغیر، مقدار معتبر برای متغیرهای مجاور وجود دارد یا نه. اینکار برای همه‌ی متغیرها انجام میشود و می‌تواند به صورت پیشرفته‌تری از فوروارد چکینگ عمل کند. این کار پیش از اینکه مقداردهی انجام شود و برای همه متغیرها انجام می‌شود، نه فقط بعد از مقداردهی. مثال (همون قبلی)

Arc consistency بررسی می‌کند که آیا برای هر مقدار از X، برای Y مقدار معتبر وجود دارد یا نه.

برای $X=1$، باید چک کنیم که آیا برای Y مقداری است که برابر 1 نباشد؟ بله $Y=2$، $Y=3$

برای $X=2$ ~ ~ ~ ~ ~ ~ ~ بله $Y=1$ ، $Y=3$

برای $X=3$ ~ ~ ~ ~ ~ ~ ~ بله $Y=1$ ، $Y=2$

در این مثال Arc consistency هیچ مقدار نامعتبری را از دامنه حذف نمی‌کند چون برای هر مقدار از X مقدار معتبر در Y هست

Ordering :

به ما کمک می‌کند که انتخاب‌های بهینه‌تری در طول جستجو داشته باشیم و فضای جستجو را به شکلی بهینه‌تر جستجو کنیم. با استفاده از تکنیک‌های مختلف ترتیب دهی متغیرها و مقادیر، می‌توانیم زمان حل مسئله‌های CSP را کاهش دهیم و احتمال رسیدن به راه حل صحیح را افزایش دهیم.

کدام متغیر بهتره توی نوبت بعدی انتخاب بشه؟

متغیری که کمترین محدودیت رو داره. این استراتژی با نام MRV هم شناخته میشه.

متغیری را انتخاب کنید که کمترین محدودیت را روی متغیرهای باقی مانده اعمال می‌کند.

حالا یک متغیر را انتخاب کردیم از کدام مقدار استفاده کنیم؟

هنگامی که یک متغیر را انتخاب کردیم و باید یک مقدار به آن تخصیص دهیم انتخاب مقادیر مختلف می‌تواند تأثیر زیادی بر دیگر متغیرهای مسئله بگذارد. یکی از استراتژی‌های معروف در این زمینه به این معناست که باید مقداری را انتخاب کنیم که کمترین تأثیر را بر دیگر متغیرها داشته باشد و کمترین محدودیت را برای آن‌ها ایجاد کند، به عبارت دیگر، این مقدار کمترین تعداد از مقادیر ممکن را از دامنه متغیرهای دیگر حذف می‌کند

SOBHAN

DATE : / / SUB :

گاهی اوقات مسئله اصلی رو میشه به چند تا زیر مسئله تقسیم کرد مثل اگر شکل چند تا مولفه همبندی داشته باشه میتونیم مسئله رابرای هر مولفه حل کنیم چون هیچ ربطی به هم ندارند.

این تقسیم بندی c متغیر داریم که میتوان آن را به زیر مسئله هایی با n متغیر تقسیم کرد.

در اینجا n تعداد کل متغیر های مسئله است و c تعداد متغیرهایی است که در هر مولفه همبند قرار دارند اگر گراف به طور کلی به m مولفه همبند تقسیم شود و در هر مولفه، تعداد متغیرها c باشد $n = m \times c$

پیچیدگی زمانی بدترین حالت: پیچیدگی زمانی به صورت $O(nd^c)$ خواهد بود که در آن:

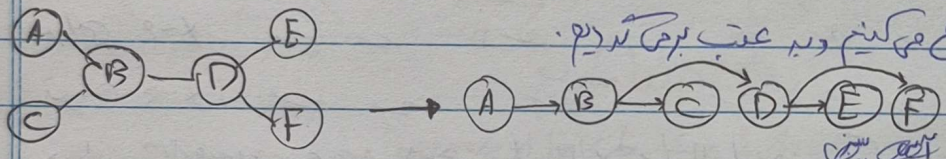
d تعداد مقادیر ممکن برای هر متغیر است (مثلا تو مثال رنگ آمیزی $\rightarrow d=3$)

پیچیدگی کلی $\rightarrow O(m \times d^c) = O(\frac{n}{c} \times d^c)$ است.

توی مسائل CSP اگه ساختار درختی داشته باشیم d^c خیلی بهتر از گراف است چون دور نداریم.

مراحل الگوریتم: انتخاب یک متغیر به عنوان ریشه: در این مرحله، یکی از متغیرها را به عنوان "ریشه" انتخاب می‌کنیم. انتخاب این ریشه مهم است زیرا از آن به بعد سایر متغیرها باید به ترتیبی مشخص درخت وار مرتب شوند.

★ بعد از برگ شروع می‌کنیم و به عقب برمی‌گردیم.

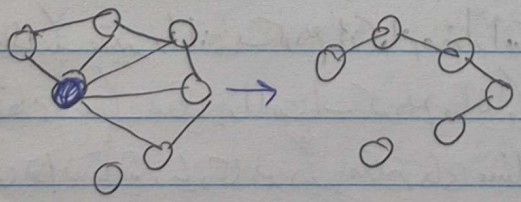


به طور خلاصه مراحل به شرح زیر هستند: ① مرحله ترتیب دهی: متغیرها را به طور ترتیبی از ریشه به برگ ها مرتب می‌کنیم ② مرحله حذف به عقب: از برگ ها به ریشه می‌رویم و با بررسی arc consistency، مقادیر غیرمعتبر را حذف می‌کنیم ③ مرحله تخصیص پیشرو: از ریشه به برگ ها حرکت کرده و مقادیر هر متغیر را از دامنه متغیر انتخاب می‌کنیم

زمان اجرا = $O(n \times d^2)$ چون برای هر کدام از Arc ها بررسی می‌کنیم

ولی اگه یکسری یال نباشه $O(d^n)$

اوس cutset: مجموعه‌ای از راس ها را حذف می‌کنیم تا گراف تبدیل به درخت شود، و به تعداد ای مولفه تبدیل میشه. $O(d^c(n-c)d^2)$



5ام جزوه CSP رو ببین

SOBHAN

DATE : / / SUB :

(Adversarial Search) این الگوریتم در محیط های رقابتی و قطعی (Deterministic) هست و هدف الگوریتم اینه که برای یک بازه محاسبه کند که چه استراتژی داشته باشد برای هر حالت.

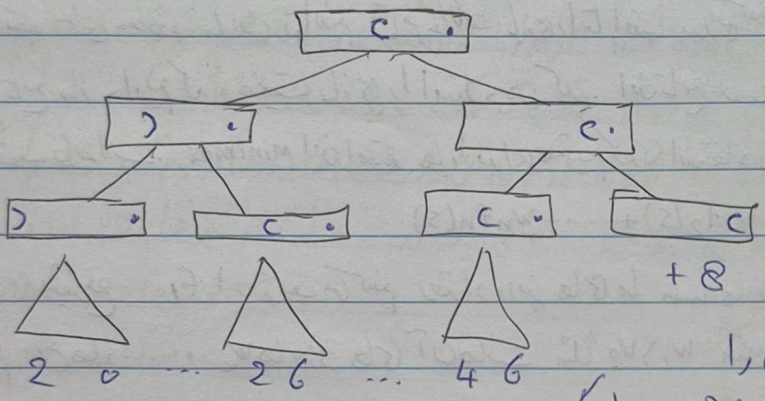
Zero-sum Games : بازی هایی که مجموع امتیازات شخص و حریف برابر با صفر است یعنی اینکه مثلاً اگر کسی 2+ امتیاز بگیرد از کار برد مقابل 2- امتیاز دریافت کرده است.

پس حرکت و امتیاز شخص به شخص دیگر وابسته هست

ولی بقیه بازی ها به صورت مستقل هستند. که این الگوریتم شامل تقریبا بازی ها الگوریتم هست

مثال: بازی یک من ← اگر یک من غذا بخورد 10+ امتیاز می گیرد ولی حرکت دیگر ۱- می گیرد.

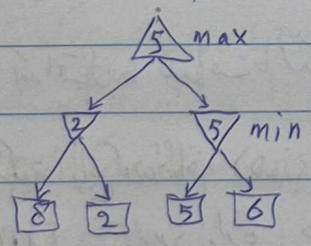
Single-Agent Trees :



برای هر است dp آن را

برابر با dp max بچه ها را در نظر میگیریم.

روش Minimax Value : توی بازی ها Adversarial برای محاسبه امتیاز ما میایم فرض میکنیم که هم ما هم حریف آپتیمال بازی میکنیم یعنی چقدر بهترین بازی ممکن را می کنیم پس اگر در درخت بازی نوبت ما باشد max بچه ها رو میگیریم و اگر نوبت رقیب باشد min بچه ها رو میگیره



این الگوریتم مثل dfs به صورت بازگشتی عمل می کند

پس لازم آن $O(b^m)$ هست و حافظه آن $O(bm)$ که تا حدی زیاد

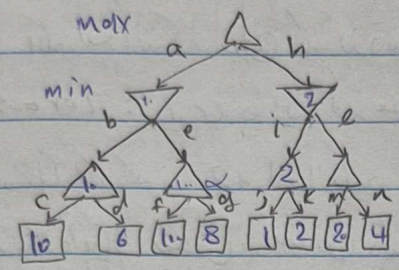
حالا سوال پیش میاد چجوری این الگوریتم رو بهینه کنیم؟

* ← باید به جای dfs از ucs یا همان dfs تا عمق محدود استفاده کنیم و از روش alpha,beta استفاده کنیم. فرض کنید α بهترین مقداری است که max در مسیر فعلی پیدا کرده است. اگر مقدار n کوچکتر از α کمتر شود، یعنی به درد چیزی است که max قبلاً دیده، پس max هرگز این مسیر را انتخاب نخواهد کرد در نتیجه می توانیم بررسی فرزندان n را متوقف کنیم و نیازی به ادامه ی جستجو نیست.

SOBHAN

DATE : / / SUB :

شرط متوقف: اگر در حین بررسی مقداری پیدا کند که لزوم (کمترین مقدار ممکن که min متوقف می‌کند) نتیجه‌ی بالا ادامه بررسی را متوقف می‌کند.



مثال:

چون ۱۰۰ > ۱۰ است پس لازم نیست g را بررسی کنیم

حال ریشه مسیر ۱۰ ، میدانیم ۲ < ۱۰ پس

لازم نیست e و دیگر بررسی نمی کنیم

تابع ارزیابی در minimax (Eralvation function)

در جستجو عمق محدود ما نمی توانیم تمام حالات بازی را تا انتها بررسی کنیم، بنابراین:

• به یک تابع نیاز داریم که وضعیت بازی را امتیاز دهی کند. این تابع به ما می گوید که یک وضعیت چقدر خوب یا بد است. minimax این امتیاز ها را برای تصمیم گیری استفاده می کند.

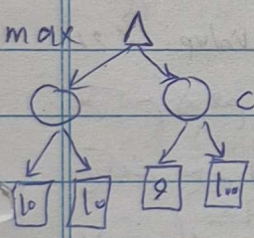
$$eval(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s)$$

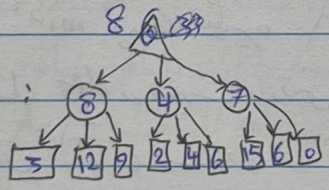
مثلاً توی شطرنج f_1 را تعریف می کنیم تعداد وزیر های ما منهای وزیرهای حریف و f_2 را میگیم رخ ها و ... و w_i ها وزن های آن حالات مثلاً w_1, w_2 میگیم چون وزیر مهمتره.

other Game Types ← Stochastic Game ← بازی هایی که قطعی نیست و شانس دخیل هست.

ExpectiMax Search ← کارهای اوقات Agent مقابل ما بهینه عمل نمی کند و شانسی عمل می کند در این صورت ما باید امید ریاضی امتیازی که میبینیم بگیریم و حساب کنیم

$$E(x) = \sum_{i=1}^{n} p_i x_i$$





Expectimax : Example

اگر احتمال ها یکنواخت باشد:

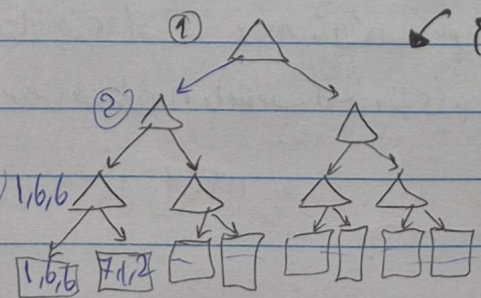
* نکته: پاتچ کردن شاخه (همون روش $\alpha\beta$) توی ExpectiMax کاربرد نداره چون شانسی هست و قطعی نیست. و اضافه در این درس ما بازی به محاسبه احتمالات نداریم و خود سوال به ما احتمال رو میده

حواست: خوشبینی بیش از حد و بدبینی بیش از حد نباید داشته باشیم. مثال:

	Adversarial Ghost	Random Ghost
Minimax Pacman	Won: 5/5 Avg. Score: 483	5/5 493
ExpectiMax Pacman	1/5 -303	5/5 503

Results from Playing 5 Games

همچنین multy-Agent هم داریم ←



SOBHAN

DATE : / / SUB :

ص ۱۶

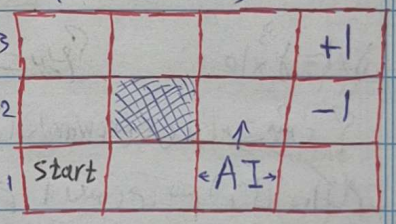
Markov Decision Processes: MDP در محیط‌های نامطمئن کاربرد دارد.

یادآوری: در محیط‌های قطعی (deterministic)، وقتی عامل به عمل انجام میده (مثلاً برو شمال)، دقیقاً همون اتفاقی که انتظار داره می‌افته. ولی در محیط‌های نامعین، ممکنه نتیجه‌ی عمل مشخص نباشه. مثلاً عامل بخواد "برو شمال" ولی به خاطر سر بخوره و بره شمال غرب یا اصلاً همون جا بمونه!

مهم‌ترین مثال این مبحث: یک محیط مربعی داریم و agent قرار یکی از خونه‌هاش زندگی میکنه و باید حرکت کنه تا به هدف برسه. / بعضی خونه‌ها دیوار دارن و عامل نمیتونه از اون‌ها عبور کنه. عامل میتونه سعی کنه به هر ۴ جهت ↑ ↓ ← → بره. ولی چون محیط نامطمئن هست حرکت‌ها همیشه اونطوری که عامل میخواد پیش نمیره مثلاً اگر قرار باشه بره شمال، ۸۰٪ احتمال میره شمال، ۱۰ غرب و ۱۰ شرق.

همچنین در هر مرحله یک پاداش کوچک میگیره که ممکنه + یا - باشه (مثلاً از -۰.۰۴)

هدف عامل: عامل باید سیاستی پیدا کنه که مجموع پاداش‌ها در طول زمان رو ماکزیمم کنه یعنی یاد بگیره توی این محیط نامطمئن: ۰ چه جهتی بره ۰ کی ریسک کنه ۰ از کدوم مسیرها پرهیز کنه.



set of state $s \in S$

set of action $a \in A$

Transition function $T(s,a,s')$

→ $P(s'|s,a)$ یعنی اگر تو استیت s باشیم و اکشن a را انجام بدیم به چه احتمالی به استیت s' میرویم (نان دترمینیستیک - راستان میره).

reward function $R(s,a,s')$ میزان پاداش

چطوری حل می‌کنیم؟ یکی از راه‌حل‌های MDP ها استفاده از الگوریتم Expectimax هست، که مثل minimax در بازی‌هاست ولی به جای "بدترین حالت"، میانگین وزنی پاداش‌ها با توجه به احتمال‌ها رو حساب می‌کنه.

یک خاصیت مهم این الگوریتم اینه که فقط به حالت فعلی نیاز داریم لازم نیست بدونیم قبلش از کجا اومدیم یا چکار کردیم.

تفاوت هدف در حل MDP: در جستجو قطعی: هدف یافتن یک برنامه بهینه (دنباله‌ای از ~~حرکت~~ شروع تا هدف) است.

در MDP، هدف یافتن یک سیاست بهینه $\pi^*: S \to A$ است یعنی برای هر وضعیت، بهترین عمل را مشخص می‌کند.

سیاست هم بدون خودمونی یعنی برای اون مسیر با توجه به هزینه قدش‌ها را به صورت بهینه رسم کنیم.

یک مثال دیگه هم زده. اون ماشینه، راحت خود تو حفره میفته.

Utility of a sequence ← یعنی اگر به عامل یه سری کار انجام بده و وارد یه سری وضعیت بشه، جمع پاداش‌هایی که از این مسیر میگیره، میشه Utility اون مسیر یا دنباله.

یک نتیجه مهم: عامل باید دنبال دنباله‌ای از اعمال باشه که جمع پاداش‌هاش (ترجیحا زودتر) بیشتر شود.

SOBHAN

DATE : / / SUB :

Discounting : تو محیط‌های MDP هدف اینکه مجموع پاداش‌ها (rewards) رو بیشینه کنیم، اما این نکته مهم هست: معمولاً بهتره پاداش رو زودتر بگیریم تا دیرتر. پس از Discounting استفاده می‌کنیم که پاداش‌های آینده، ارزش کمتری نسبت به حال دارن.

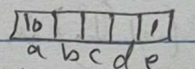
فرمول ایجی: اگر ضریب رو با γ (گاما) نشون بدیم (عددی بین ۰ و ۱) ارزش پاداش‌ها این شکلی محاسبه میشه:

Reward in one step → $\gamma \times$ reward
" " two " → $\gamma^2 \times$ reward
" " n " → $\gamma^n \times$ reward

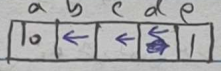
Reward now → همون مقدار خودش (بدون تغییر)

نتیجه: هر چی γ بزرگتر باشه مثلاً (۰٫۹)، آینده مکدر تر هستیم. هر چی γ کوچکتر باشه (مثلاً 0.5)، مقدار تردیک‌ها برامون مهمه.

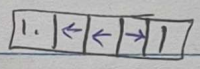
مثال: $\gamma = 0.5$ $\quad U([1,2,3]) < U([3,2,1])$ $\quad U([1,2,3]) = 1\times1 + 0.5\times2 + 0.25\times3$

مثال خوب: 

در این مثال در خانه‌های a و e اجازه خارج شدن داریم و تو دقیقه خانه یا میتونیم بریم راست یا چپ و محیط ترمینیستیک هست حالا فرض کنیم تو کی خونه‌ای هستیم.



سوال ۱: برای $\gamma = 1$ سیاست چطور باید باشه؟



سوال 2: برای $\gamma = 0$ ؟

سوال 3: برای چه مقداری از γ فرقی نداره کدام یک رو بکدوم سمت بریم؟ $\gamma^2 = 0.1 \Rightarrow \gamma = 0.31$ $\quad \gamma \times 1 = \gamma^3 \times 10$

Problem: اگه بازی یا محیط هیچ وقت تموم نشه، آیا جایزه‌ها (rewards) بی نهایت میشن؟

★ دو تا روش داریم: ① Finite Horizon → یه حداکثر تعداد گام T تعیین می‌کنیم (مثلاً ۱۰۰ گام تا پایان عمر)

② Discounting: استفاده از ضریب تنزیل با شرط $\gamma < 1$ که جمع بی‌نهایت $r_0 + r_1\gamma + r_2\gamma^2 + r_3\gamma^3 + \cdots = \frac{R_{max}}{1-\gamma}$ همگرا میشه

$V^*(s)$: بیشترین مقدار انتظاری از پاداش کلی که میتوان از حالت s شروع کرد و از آن به بعد به بهترین نحو بازی کرد. ساده: اگر از حالت s شروع کنم و همیشه بهترین تصمیم‌ها رو بگیریم در مجموع چقدر پاداش خواهیم گرفت؟

$Q^*(s,a)$: اگر الان توی حالت s هستم و تصمیم دارم عمل a رو انجام بدم و بعدش همیشه بهترین تصمیم‌ها رو بگیرم در مجموع چقدر پاداش می‌گیرم؟ این کمیت به ما کمک می‌کنه تصمیم بگیریم که در هر عمل در یک حالت بهتره.

$\pi^*(s)$: در حالت s چه عملی رو باید انجام بدم تا به بیشترین پاداش ممکنه برسیم؟ این همون چیزیه که دنبالشیم!

نحوه محاسبه مقدارهای مراحل: فرمول‌های پایین

① $Q^*(s,a)$ $\max_a$ $= V^*(s)$ اگه در حالت s هستی، بهترین کاری که می‌تونی بکنی چیه؟ همونو انتخاب کن

② $T(s,a,s')\left[R(s,a,s') + \gamma \cdot V^*(s')\right]$ $\sum$ $= Q^*(s,a)$
1- با عمل a از s حرکت می‌کنی
2- با احتمال $T(s,a,s')$ به هر s' میری
3- پاداش میگیری $R(s,a,s')$ و از اونجا به بعد $V^*(s')$ هست که باید محاسبه بشه

فرمول کلی: $V^*(s) = \max_a \sum_{s'} T(s,a,s)\left[R(s,a,s') + \gamma V^*(s')\right]$ $\quad O(s^2A)$

SOBHAN

مثال صفحه آخر خیلی خوبه MDP رو [illegible] بیین

معادله Bellman : مشخص میکنه که مقدار بهینه هر حالت، برابر با بیشترین مقداریه که از انتخاب یک اکشن میتونه به دست بیاد، با در نظر گرفتن پاداش فوری و مقدار حالت بعدی

فرمولش: $V^*(s) = \max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma V^*(s')\right]$

الگوریتم Value Iteration : این الگوریتم به صورت تکراری مقدار حالت ها رو به روزرسانی میکنه تا جایی که این مقادیر بهینه نزدیک بشن. $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma V_k(s')\right]$

Policy Methods : در این روش ها به جای تمرکز بستن روی مقدار بهینه حالت ها، تمرکز می کنیم روی سیاست. یعنی اینکه در هر حالت چه کاری انجام بدیم. (همون عامل ها)

Policy Evaluation : هدف: اگه یه سیاست $\pi(s)$ مشخص داشته باشیم (یعنی بدونیم تو هر حالت چه کاری انجام می دیم)، می خواهیم مقدار $V^\pi(s)$ برای همه حالت ها رو حساب کنیم.

Fixed Policy : اگه سیاست $\pi(s)$ رو ثابت کنیم، درخت تصمیم گیری ساده تر میشه. به جای اینکه توی هر حالت بین همه اکشن ها بیشینه بگیریم (مثل Value Iteration)، مقدار همون اکشن داده شده توسط $\pi(s)$ رو در نظر میگیریم. یعنی به جای اینکه درخت Expectimax باشه (با بیشینه گیری که هر لایه عوض میشه)، یه درخت با یک مسیر مشخص داریم.

میخواهیم بدونیم اگه از حالت s شروع کنیم و سیاست ثابت π رو دنبال کنیم، حدود پاداش کل انتظار میره چقدر باشه.

فرمول بازگشتی: $V^\pi(s) = \sum_{s'} T(s,\pi(s),s')\left[R(s,\pi(s),s') + \gamma V^\pi(s')\right]$ ← $O(s^2)$

Policy Extraction (استخراج سیاست):

هدف: مقدارهای بهینه $V^*(s)$ در اختیار داریم می خواهیم در هر حالت بهترین اکشن رو انتخاب کنیم، برای این کار از یک کار expectimax استفاده می کنیم $\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s')\left[R(s,a,s') + \gamma V^*(s')\right]$

مثال ساده از این فرمول: فرض کن یه محیط MDP کوچک داریم با 2 حالت: s_1, s_2 و دو اکشن a_1, a_2

و فرض کنیم: $V^*(s_1) = 10$, $V^*(s_2) = 5$ ، تابع انتقال به این صورت هست:

$s_1 \xrightarrow{a_1}$ {0.5 به s_2 ، 0.5 به s_1} پاداش = 2 / $s_1 \xrightarrow{a_2}$ {1 به s_1} پاداش = 0

$s_2 \xrightarrow{a_1, a_2}$ {1 به s_2} پاداش = 1 ، $\gamma = 0.9$

هدف: $\pi^*(s_1), \pi^*(s_2)$

s_1: action: a_1 → $Q(s_1,a_1) = 2 + 0.9 \cdot [0.5 \cdot V^*(s_1) + 0.5 \cdot V^*(s_2)] = 8.75$

action: a_2 → $Q(s_1,a_2) = 0 + 0.9 \cdot (1.0 \cdot V^*(s_1)) = 0.9 \times 10 = 9$

پس → $\pi^*(s_1) = a_2$

s_2: $Q(s_2,a) = 1 + 0.9 \cdot V^*(s_2) = 1 + 0.9 \cdot 5 = 5.5$

$\pi^*(s_2) = a_1$ or a_2 (both are optimal)

SOBHAN

CamScanner

Poling Iteration : این یه روش دیگه‌ست که بابت بهتری نسبت به روش قبلی حل می‌کند که دو مرحله داره:

مرحله ① : ابتدا یک سیاست ثابت در نظر میگیریم (مثلا بین بیاد)

مرحله ② : این سیاست را ارزیابی میکنیم. (با استفاده از معادلات بدست آمده) در هر حالت عملی که بیشترین مقدار پاداش را به دست می‌دهد انتخاب می‌شود سپس سیاست جدید جایگزین قبلی میشه

با تکرار این دو مرحله زمانی میرسیم که سیاست دیگر تغییر نکند.

Probability and bay's Net همون چیزای بردیس آماره که اکثرشو نفهمیدم

Normalization Trick

$P(T,W)$

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

$P(C,W)$

T	W	P
cold	sun	0.2
cold	rain	0.3

W	P
sun	0.4
rain	0.6

$$P(W=s|T=c) = \frac{P(W=s,T=c)}{P(T=c)} = \frac{P(W=s,T=c)}{P(W=s,T=c)+P(W=r,T=c)} = \frac{0.2}{0.2+0.3} = 0.4$$

$$P(W=r|T=c) = \frac{P(W=r,T=c)}{P(T=c)} = \frac{P(W=r,T=c)}{P(W=s,T=c)+P(W=r,T=c)} = \frac{0.3}{0.2+0.3} = 0.6$$

Inference by Enumeration (استنتاج با شمارش کامل) یعنی برای محاسبه یک احتمال (مثلا شرطی) بیاییم همه حالات ممکن رو بشماریم و جمع بزنیم (مثل کاری که با جدول توزیع مشترک انجام میدیم)

مشکل: زمان و حافظه زیاد مصرف می‌کند. $O(d^n)$ نمایی.

قانون زنجیری: $P(x_1,x_2,x_3) = P(x_1)P(x_2|x_1)P(x_3|x_1,x_2)$

$$P(x_1,x_2,\dots,x_n) = \prod_i P(x_i|x_1,\dots,x_{i-1})$$

قانون بیز:

$$P(x,y) = P(x|y)P(y) = P(y|x)P(x) \longrightarrow P(x|y) = \frac{P(y|x)}{P(y)}P(x)$$

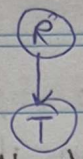
مثال: $P(+s|-m) = 0.01$, $P(+s|+m) = 0.8$, $P(+m) = 0.0001$

$$P(+m|+s) = \frac{P(+s|+m)P(+m)}{P(+s)} = \frac{P(+s|+m)P(+m)}{P(+s|+m)P(+m)+P(+s|-m)P(-m)} = \frac{0.8\times0.0001}{(0.8\times0.0001)+(0.01\times0.9999)}$$

DATE : / / SUB :

ص 23

مثال شبکه بیزی :



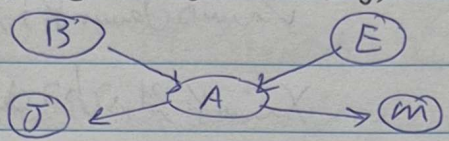
Variables: R: it rains, T: there is traffic

مثال زنگ خطر دزدی (Burglar Alam) : ماکه زنگ خطر داریم که در اثر دزدی یا زلزله فعال میشود و اگر همزمان سیم زنگ نزنن

Variables: Burglary (B) - آیا دزدی خانه داره Earthquake (E) - آیا زلزله خفیف رخ داده

Alarm (A) - آیا زنگ خطر فعال شده JohnCalls (J) آیا جان زنگ زده Marycalls (M) - آیا مری زنگ زده؟

ساختار کلی →



نکات خیلی مهم: ① $P(J|A,B) = P(John|Alarm)$

یعنی اگر زنگ رو شنیده باشی، دیگه دونستن اینکه دزدی شده یا نشده تأثیری روی اینکه جان زنگ میزنه یا نه، نداره

* پس J و B نسبت به هم مستقل شرطی هستن وقتی A رو بدونن

② $P(M|A,J) = P(M|A)$

یعنی وقتی میدونی زنگ خطر فعال شده، اینکه جان زنگ زده یا نه تأثیری در احتمال زنگ زدن مری نداره.

* پس J و M مستقل شرطی از هم هستن وقتی A رو بدونن

③ $P(B|A,E) \neq P(B|A)$

یعنی اگر فقط زنگ رو شنیدی، احتمال دزدی به خیلی بالا میره ولی اگر بدونی زلزله هم بوده احتمال دزدی کمتر میشه چون ممکنه زنگ به خاطر زلزله بوده باشه.

* پس B و E مستقل شرطی از هم نیستن وقتی A رو بدونی

④ $P(A|B,J) \neq P(A|B)$

یعنی اگر بدونی دزدی شده، یک احتمال بالایی زنگ وجود داره، ولی اگر بدونی جان هم زنگ زده، احتمال فعال بودن زنگ تغییر میکنه

* پس J و A نسبت به هم مستقل شرطی نیستن وقتی فقط B رو بدونی

قواعد کلی:

فرزندان مستقل از اجداد وقتی پدر رو بدونی ← J مستقل از B وقتی A رو بدونی

خواهر و برادر مستقل از هم وقتی پدر رو بدونی ← J و M مستقل از هم وقتی A رو بدونی

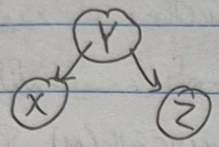
والدین مستقل از هم نیستن وقتی فرزندو بدونی ✗ B و E مستقل نیستن وقتی A رو بدونی

کلا مستقل از فرزندانش نیست وقتی فقط والدینش رو بدونی ✗ A مستقل از J نیست وقتی فقط B رو بدونی

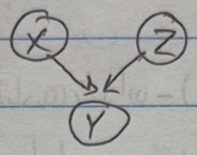
SOBHAN

مثال ساده‌تر: کودک = J و دزدگیر = A و فرزندان = ∅ و غیرخانوادگانش: M و E و B

طبق قانون: اگر A رو بدونیم آنگاه J مستقل از M و B و E هست.



X و Z مستقل نیستند X



X و Z مستقل هستند ✓

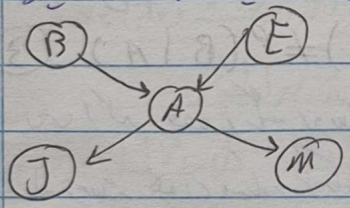
X و Z مستقل شرطی در اثر Y نیستند X

X و Z مستقل شرطی در اثر Y هستند ✓

شبکه بیزی یعنی چی؟ یک گراف جهت دار بدون چرخه (DAG) و هرکدوم نماینده‌ی یک متغیر تصادفه (مثلاً A، J، B، ...) یال جهت دار (→) نشون میده که یک متغیر روی متغیر دیگه تأثیر مستقیم داره. یک جدول احتمال شرطی (CPT) برای هرکدوم که نشون میده احتمال متغیر با توجه به والدینش چیه

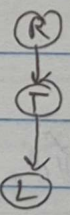
فرمول:
$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^{n} P(x_i \mid Parents(x_i))$$

یعنی برای بدست آوردن احتمال هر ترکیب کامل از متغیرها، کافیه احتمال هر متغیر رو با توجه به والدینش ضرب کنیم.



$$P(+b, -e, +a, -j, +m) = P(+b)P(-e)P(+a \mid +b, -e) \cdot P(-j \mid +a)P(+m \mid +a)$$

در این مثال:



برای بدست آوردن $P(L)$ یک راه این هست که $P(R,T,L)$ رو بدست بیاریم و بعد از آن $P(L)$ بدست بیاریم یعنی اول $P(r).P(t|r)$ میکنیم تا $P(r,t)$ بدست بیاد بعد $P(r,t)$ رو از $P(L|t)$ ضرب می‌کنیم تا $P(R,t,L)$ بدست بیاد بعد با sum مسئله همه چی رو در آورد.

روش دیگه: ابتدا $P(r,t)$ رو بدست می‌آوریم و بعدش با sum $P(t)$ رو در میاریم بعدش با ضرب کردن $P(t).P(L|t)$ رو بدست می‌آوریم و با sum $P(L)$ رو در میاریم

مثال مهم: فرض کنیم می‌خواهیم $P(B|j,m)$ رو بدست بیاریم که این معادل این هست اول $P(B,j,m)$ رو بدست بیاریم

$$P(B,j,m,A,E) = P(B).P(E).P(A|B,E).P(j|A).P(m|A)$$

متغیرهای A و E رو حذف کنیم / choose(A) $P(A|B,E)$ $P(j|A)$ $P(m|A)$ $\rightarrow P(j,m,A|B,E) \xrightarrow{\Sigma} P(j,m|B,E)$

choose(E) $P(E)$ $\rightarrow P(j,m|B,E) \rightarrow P(j,m,E|B) \xrightarrow{\Sigma} P(j,m|B) \xrightarrow{P(B)} P(j,m,B) \rightarrow$

SOBHAN

Hidden Markov models وقتی با داده هایی روبرو هستیم که در طول زمان یا مفنا تغییر میکنن (مثل صدا، حرکت ربات، وضعیت بیمار)، باید مدل هایی داشته باشیم که این "توالی" رو درک کنن و پیش بینی درست کنن، این همون جاییه که مدل هایی مثل HMM وارد ماجرا میشن.

اجزای اصلی مدل:

① State : وضعیت واقعی سیستم در هر لحظه که مثلاً با متغیر X نشان میدهیم. (مثلاً آب‌وهوا)

② observation : چیزی که میبینیم مثلاً مردم چتر دارن یا نه

③ Transition Probability : احتمال جابجایی بین حالت‌ها، مثلاً احتمال اینکه فردا بارون باشه اگر امروز آفتابی

④ Emission Probability : احتمال دیدن یه مشاهده در یک حالت خاص، مثلاً احتمال اینکه مردم چتر داشته باشن وقتی بارونیه

⑤ initial Probability : احتمال شروع از هر حالت در لحظه اول.

★ تفاوت با MDP اینه که اینجا هیچ عملی انجام نمیدیم فقط سیستم رو مشاهده می کنیم.

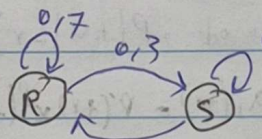
Hmm، وضعیت فعلی تمام اطلاعات لازم برای پیشبینی آینده رو داره. یعنی اگر وضعیت الان رو بدونیم دیگه نیازی نیست بدونیم دیروز یا پریروز چی بوده. گذشته و آینده نسبت به هم مستقل اند اگر وضعیت فعلی داده شده باشه.

$X_1 \to X_2 \to \cdots \to X_n$ $P(X_t | X_{t-1})$

مثال: States: $X = \{rain, sun\}$ ① sun 1.0 توزیع اولیه

CPT $P(X_t | X_{t-1})$:

X_{t-1}	X_t	$P(X_t \mid X_{t-1})$
sun	sun	0,9
sun	rain	0,1
rain	sun	0,3
rain	rain	0,7



$P(X_2 = sun) = \sum_{x_1} P(x_1, X_2 = sun) = \sum_{x_1} P(X_2 = sun | x_1) P(x_1)$ بعد از یک مرحله چه اتفاقی می افته؟

$P(X_2 = sun) = P(X_2 = sun | X_1 = sun) P(X_1 = sun) + P(X_2 = sun | X_1 = rain) P(X_1 = rain) = 0,9 \times 1 + 0,3 \times 0 = 0,9$

$P(X_2 = rain) = P(X_2 = rain | X_1 = sun) P(X_1 = sun) + P(X_2 = rain | X_1 = rain) P(X_1 = rain) = 0,1 \times 1 + 0,7 \times 0 = 0,1$

الگوریتم mini-forward ← میخواهیم بفهمیم در روز t، احتمال اینکه سیستم در حالت خاصی باشه چقدره؟

$P(X_1) = known$ $P(X_t) = \sum_{x_{t-1}} P(X_{t-1}, X_t) = P(X_t | X_{t-1}) P(X_{t-1})$ یعنی میخواهیم $P(X_t)$ رو حساب کنیم.

مثالش همین مثال بارانی ✓

$\langle 1.0, 0.0 \rangle$ $P(X_1)$ → $\langle 0.9, 0.1 \rangle$ $P(X_2)$ → $\langle 0.84, 0.16 \rangle$ $P(X_3)$ → $\langle 0.804, 0.196 \rangle$ $P(X_4)$ → $\langle 0.75, 0.25 \rangle$ $P(X_\infty)$

توزیع ایستا (Stationary Distribution): وقتی زنجیره مارکوف رو برای مدت طولانی اجرا کنیم، معمولاً مهم نیست از کجا شروع کنیم (initial state)، چون بعد از چند گام، احتمال بودن در هر حالت به یک توزیع ثابت میرسه.

مثلاً تو همون مثال قبلی داریم: $P_\infty(sun) = P(sun|sun)P_\infty(sun) + P(sun|rain)P_\infty(rain)$

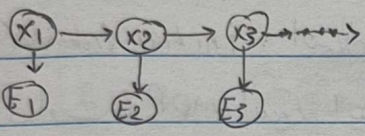
$P_\infty(Rain) = P(rain|sun)P_\infty(sun) + P(rain|rain)P_\infty(rain)$

$P_\infty(sun) = 0.9P_\infty(sun) + 0.3P_\infty(rain) \longrightarrow P_\infty(sun) = 3P_\infty(rain)$

$P_\infty(rain) = 0.1P_\infty(sun) + 0.7P_\infty(rain) \longrightarrow P_\infty(rain) = \frac{1}{3}P_\infty(sun)$

Also: $P_\infty(sun) + P_\infty(rain) = 1 \longrightarrow P_\infty(sun) = \frac{3}{4}$, $P_\infty(rain) = \frac{1}{4}$

HMM: این شامل دو بخش هست ① زنجیره مارکوف پنهان روی حالت‌ها: حالت واقعی سیستم (مثل حال هوای واقعی) که مستقیم قابل دیدن نیست. ② مشاهده: در هر گام، خروجی‌ای مبتنی بر حالت واقعی داده میشود.



مثال: سه روز پشت سر هم داریم و فقط مشاهدات رو دیدیم: $u_1 = +u$ چتر دارد، $u_2 = +u$ چتر دارد، $u_3 = -u$ چتر ندارد

میخواهیم با الگوریتم Forward محاسبه کنیم: $P(U_1=+u, U_2=+u, U_3=-u)$

فرضیات: توزیع اولیه: $P(R_1=+r) = P(R_1=+r) = 0.5$

$P(R_t=+r | R_{t-1}=-r) = 0.3$, $P(R_t=+r | R_{t-1}=+r) = 0.7$: Transition

$P(-u|-r) = 0.8$, $P(-u|+r) = 0.1$, $P(+u|-r) = 0.2$, $P(U_t=+u | R_t=+r) = 0.9$: Emission

حل: روز ①: $a_1(+r) = P(+r) \times P(+u|+r) = 0.5 \times 0.9 = 0.45$

$a_1(-r) = P(-r) \times P(+u|-r) = 0.5 \times 0.2 = 0.1$

روز ②: $a_2(+r) = [a_1(+r) \times P(+r|+r) + a_1(-r) \times P(+r|-r)] \times P(+u|+r) = 0.315$

$a_2(-r) = [\sim \sim] = 0.041$

روز ③: $a_3(+r) = [a_2(+r) \times P(+r|+r) + a_2(-r) \times P(+r|-r)] \times P(-u|+r) = 0.1022965$

$a_3(-r) = [\sim \sim] = 0.09748$

جمع نهایی $\Rightarrow P(U_1=+u, U_2=+u, U_3=-u) = a_3(+r) + a_3(-r) = 0.1204$

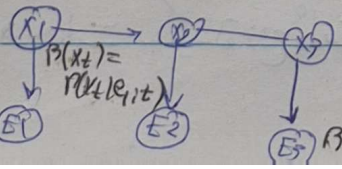
Filtering/monitoring $\leftarrow P(X_t | e_1, e_2, \dots, e_t) = B_t(X)$ که $B_t(X)$ همان حالت باور در زمان t

یعنی احتمال اینکه سیستم در حالت X_t باشد با توجه به همه شواهد و مشاهدات $e_1, \dots, e_t$ تا آن زمان

Filtering $\longrightarrow P(X_t | e_{1:t})$, Prediction $\longrightarrow P(X_{t+k} | e_{1:t})$

Smoothing $\longrightarrow P(X_k | e_{1:t})$ Most like explanation $\longrightarrow \max$

$\operatorname{argmax}_{u_{1:t}} P(x_{1:t} | e_{1:t})$



$P(x_t) = P(x_t | e_{1:t})$

$P(u_{1:t} | e_{1:t})$

SOBHAN

$$P(X_1|e_1) = \frac{P(X_1,e_1)}{\sum_{x_1} P(x_1,e_1)} = \frac{P(e_1|X_1)P(X_1)}{\sum_{x_1} P(e_1|x_1)P(x_1)}$$

$B_t(X)$ یعنی احتمال اینکه سیستم در لحظه t در وضعیت X باشد، با توجه به تمام شواهدی که تا الان (تا لحظه t) دیده‌ایم.

وقتی فقط زمان میگذرد و مشاهده جدیدی نداریم، برای بروزرسانی باور نسبت به وضعیت جدید از رابطه $P(X_{t+1}|e_{1:t}) = \sum P(X_{t+1}|x_t).P(x_t|e_{1:t})$ استفاده می‌کنیم که به این فرآیند می‌گویند.

Time elapse Update

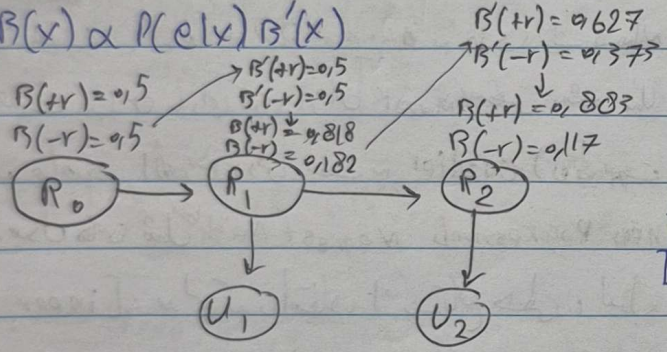
$$B'(X_{t+1}) = \sum_{x_t} P(X'|x_t)\,B(x_t)$$

حالا اگر واسه $B'(X_{t+1}) = P(X_{t+1}|e_{1:t})$

$P(X_{t+1}|e_{1:t+1}) = P(X_{t+1}, e_{t+1}|e_{1:t}) / P(e_{t+1}|e_{1:t}) \propto$ ~~$P(X_{t+1}|e_{1:t})$~~

$\Rightarrow \alpha \rightarrow P(e_{t+1}|X_{t+1})\,P(X_{t+1}|e_{1:t})$

$B(X) \propto P(e|X)\,B'(X)$



مثال:

R_t	R_{t+1}	$P(R_{t+1}\|R_t)$
+r	+r	0,7
+r	-r	0,3
-r	+r	0,3
-r	-r	0,7

R_t	U_t	$P(U_t\|R_t)$
+r	+u	0,9
+r	-u	0,1
-r	+u	0,2
-r	-u	0,8

After seeing Umbrella_1 = +u

$B'_1(+r) = 0,7 \cdot 0,5 + 0,3 \cdot 0,5 = 0,5$

$B'_1(-r) = 0,3 \cdot 0,5 + 0,7 \cdot 0,5 = 0,5$

$B(+r) = 0,9 \times 0,5 = 0,45$

$B(-r) = 0,2 \times 0,5 = 0,1$

normalize

$B_1(+r) = \frac{0,45}{0,55} = 0,818$

$B_2(-r) = 0,182$

After seeing Umbrella_2 = +u

$B'_2(+r) = 0,7 \times 0,818 + 0,3 \cdot 0,182 = 0,627$

$B'_2(-r) = 0,3 \times 0,818 + 0,7 \times 0,182 = 0,373$

$B_2(+r) = 0,9 \cdot 0,627 = 0,5643$

$B_2(-r) = 0,2 \cdot 0,373 = 0,0764$

norm

$B_2(+r) = 0,883$

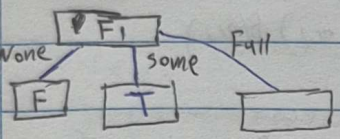
$B_2(-r) = 0,117$

ML به زبان ساده یعنی ما به جای اینکه دقیقاً به کامپیوتر بگیم چطور کاری رو انجام بده، داده‌هایی بهش میدیم و میذاریم خودش الگوها رو یاد بگیره

Naive Bayes classifier ← طبقه‌بندی بیز ساده ← فرمول قانون بیز: $P(Y|X) = \frac{P(X|Y).P(Y)}{P(X)}$

که Y: label و X: فیچر اگر فیچرها مستقل باشند ← $f(x) = \arg\max_y P(y) \prod_d P(x_d|y)$

تعریف Decision Tree: مدلی برای طبقه‌بندی که به صورت درختی از تصمیم‌ها عمل می‌کند هر گره داخلی (Node) یک ویژگی را بررسی می‌کند و هر شاخه (Edge) یک مقدار از آن ویژگی را نشان میدهد و برگ‌ها هم کلاس خروجی را مشخص می‌کنند.



مثال ساده: (زیاده حتماً خیلی ساختار در 15 pdf 14 رویس ✓

Nearest Neighbor classifier (NN) ← طبقه‌بندی نزدیک‌ترین همسایه

مدلی ساده و بدون نیاز به آموزش! برای یک ورودی جدید، برچسب نزدیک‌ترین نقطه آموزشی را به آن اختصاص می‌دهد

ساختاری که نیاز داریم یک تابع فاصله هست مثل فاصله اقلیدسی و...

طبقه‌بندی K نزدیک‌ترین همسایه ← به جای یک همسایه، K تا از نزدیک‌ترین نقاط آموزشی را پیدا می‌کند.

برچسب نهایی با رای‌گیری از این K همسایه مشخص میشود. مثلاً اگر K=3 باشد و همسایه‌ها برچسب‌های A, A, B باشند پیش‌بینی A = A

کد واضحه الگوریتم دومی نسبت به outlier (نقطه پرت) مقاوم هست

در مدل‌هایی مثل Nearest Neighbor یا Linear Regression وجود outlier میتونه نتایج رو خراب کنه

Linear classifier: مدلی که با استفاده از یک تابع خطی، لیبل‌ها رو از هم جدا میکنه فرمول کلی:

$$f(x) = \text{sgn}(w_1x_1 + w_2x_2 + \dots + x_Dw_D + b) = \text{sgn}(x \cdot w + b)$$

w = بردار وزن‌ها / x = بردار ویژگی‌ها / b = بایاس / sgn: تابع علامت (خروجی فقط 1+)

چرخه Experimentation در آزمایش‌های ML = ① آموزش مدل ← مدل روی داده‌های آموزشی، پارامترها (مثل وزن) رو یاد میگیره cycle

② validation set ← تنظیم ابرپارامترها ← مثل تعداد همسایه‌ها در KNN، عمق درخت، نرخ یادگیری و...

③ test set ← ارزیابی نهایی مدل

هدف Generalization: می‌خواهیم مدلی بسازیم که روی داده‌های جدید (ناشناخته) خوب جواب بده، نه فقط روی داده‌هایی که دیده

overfitting: وقتی مدل روی داده‌های آموزش خیلی خوب عمل کنه ولی روی داده‌های جدید (تست) ضعیف باشه / چون ممکنه الگوهای بی‌ربط یا نویز هم یاد گرفته باشه

unsupervised learning (یادگیری بدون نظارت): ما فقط داده‌های بدون برچسب داریم و میخوایم ساختاری پنهان توی داده‌ها پیدا کنیم

اهداف رایج: ① خوشه‌بندی ② کاهش ابعاد ③ تخمین چگالی ④ کوانتیزاسیون

$P(A,B) = P(A|B)P(B) = P(B|A)P(A)$ قانون بیز: (Naive_bayes)

$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

مثال: احتمال باران آمدن 0,014 میباشد، فرض کنیم هواشناس پیشبینی کرده که فردا باران میبارد میدانیم وقتی باران ببارد هواشناس 0,9 درست میگه و اگه باران نیاد هواشناس باز میگه بارون میاد حالا با توجه به اینکه هواشناس گفته فردا باران میاد احتمال اینکه واقعاً بیاد؟

$P(rain|Predict) = \frac{P(Predict|rain)P(rain)}{P(Predict)} = \frac{P(Predict|rain)P(rain)}{P(Predict|rain)P(rain)+P(Predict|\neg rain)P(\neg rain)}$

$= \frac{0,9 \times 0,014}{0,9 \times 0,014 + 0,1 \times 0,986} = 0,111$

Classification: در طبقه بندی، ما میخواهیم براساس ورودی x، یک برچسب (Label) یا کلاس y را پیشبینی کنیم. مثال: ① ورودی: یک ایمیل / خروجی: اسپم یا عادی و...

Naive bayes: ساده ← چون فرض میکنه کدوم ویژگی ها از هم مستقل اند. یعنی اگر یک ایمیل شامل چند کلمه است. فرض میکنه که احتمال وجود هر کلمه مستقل از کلمات دیگر است این فرض در واقع معمولاً درست نیست اما در عمل نتایج خوبی میدهد.

مثال: اگر دانسته باشیم $P("win"|spam) = 0,6$, $P("free"|spam) = 0,7$, $P(spam) = 0,2$

$P("money"|spam) = 0,8$,

$P(spam|free, win, money) \propto P(spam) \cdot P("free"|spam) \cdot P("win"|spam) \cdot P("money"|spam)$

$= 0,2 \cdot 0,7 \cdot 0,6 \cdot 0,8 = 0,0672$

$P(not\ spam|free, win, money) \propto 0,8 \cdot 0,1 \cdot 0,5 \cdot 0,1 = 0,004$

$model: P(Y|W_1 \dots W_n) = P(Y) \prod_i P(W_i|Y)$

برآورد پارامتر با (maximum Likelihood): فرض کن یه کیسه داریم که توش مهره های قرمز و آبی دارن

پارامتر θ رو تعریف میکنیم: احتمال این که قرمز دربیاد $P(red|\theta)$ / مثلاً 3 بار برداشتیم red ✓ red ✓ blue ✗

$\hat{\theta}_{MLE} = \arg\max_{\theta} P(\text{مشاهدات}|\theta)$ → $P(red, red, blue|\theta) = \theta \cdot \theta \cdot (1-\theta) = \theta^2(1-\theta)$

حالا max اینو با مشتق گیری پیدا میکنیم ← $\theta = \frac{2}{3}$

اگه مشاهدات زیاد بود میتونیم از Log کمک بگیریم ← $Ln\ P = Ln(\theta^2(1-\theta))$

$\frac{d}{d\theta} \rightarrow \frac{2}{\theta} - \frac{1}{1-\theta} = 0 \rightarrow \theta = \frac{2}{3}$ ✓ (تو آمار هم بود)

هدف اصلی یادگیری ماشین این نیست که مدلی بسازیم که نه فقط روی داده های آموزش، بلکه روی داده های جدید (داده های تست) هم عملکرد خوبی داشته باشد.

Overfitting : وقتی مدل خیلی زیاد روی داده های آموزش تنظیم میشه؛ تمام آموزش خیلی کمه (حتی صفر) ولی روی داده های جدید عملکرد ضعیف داره چون فقط حفظ کرده، نه اینکه الگو رو یاد بگیره

Underfitting : وقتی مدل خیلی ساده ست و حتی توی داده های آموزش رو هم خوب یاد نگرفته: هم روی آموزش و هم روی تست عملکرد ضعیف / مثلاً یک مدل خطی ساده برای داده های خیلی پیچیده

underfitting ← مدل خیلی ساده ← مدل پیچیده‌تر انتخاب کن

overfitting ← مدل بیش از حد قانون زده ← Regularization / smoothing

در مدل هایی که از شمارش مستقیم استفاده می‌کنن (مثل نایو بیز) اگر در داده های آموزش یک کلمه (یا رخداد) اصلاً ظاهر نشده باشه، احتمالش صفر در نظر گرفته میشه.

راه حل Laplace smoothing : فرض کن هر حالت ممکن رو یکبار بیشتر از واقعیت دیدی پس حتی اون هایی که اصلاً ندیدی، فرض کن یکبار دیده شدن.

$$P(w|c) = \frac{count(w,c)+1}{total\ words\ in\ c + V}$$

که اضافه کردن V باعث میشه احتمال صحیح نباشه

اضافه کردن V به مخرج (اندازه واژگان) باعث میشه کل احتمال همچنان نرمال بمونه.

$$K\text{-Smoothing} \rightarrow \frac{count(w,c)+k}{total\ words\ in\ c + k.V} = P(w|c)$$

$$P_{LAP}(x) = \frac{c(x)+1}{\sum_x [c(x)+1]} = \frac{c(x)+1}{N+|x|}$$

$$P_{LAP}(x) = \frac{c(x)+k}{N+k|x|}$$

توی مثال نمون: a b b

$$P_{ML}(x) = \left\langle \frac{2}{3}, \frac{1}{3} \right\rangle$$

$$P_{LAP_1}(x) = \left\langle \frac{3}{5}, \frac{2}{5} \right\rangle$$

$$P_{LAP_{100}}(x) = \left\langle \frac{102}{205}, \frac{101}{203} \right\rangle$$

Desition Tree تعریف کلی: درخت تصمیم یکی از مدل های یادگیری نظارت شده (Supervised Learning) است که برای طبقه بندی (classification) و رگرسیون (Regression) استفاده می‌شود. مدل به صورت یک ساختار درختی است که هر گره داخلی (node) یک شرط داره و هر برگ ها (leaves) پیش بینی نهایی انجام میشود.

مراحل ساخت درخت تصمیم: ① انتخاب بهترین ویژگی برای تقسیم (split) بهترین ویژگی، ویژگی است که بیشترین کاهش در بی نظمی داده ها را ایجاد کند. ② معیار انتخاب ویژگی ← معیارهای معروف: Entropy (آنتروپی) / Gini Index (شاخص جینی)

(کلاس/مقدار)

DATE : / / SUB :

صاحبی

تعریف آنتروپی: آنتروپی میزان بی نظمی یا عدم قطعیت در داده هاست.

$$H(\langle P_1,\dots,P_n\rangle) = Ep \log_2 \frac{1}{P_i} = \sum_{i=1}^{n} -P_i \log_2 P_i$$

که P_i احتمال کلاس i در مجموعه ی هست.

* وقتی درخت بیش از حد یاد میگیرد (overfitting): درخت های خیلی بزرگ ممکن است به نویز داده حساس شوند. / راه حل: هرس کردن (Pruning) یا محدود کردن عمق درخت

مثال آنتروپی: تو همون مثال معروف اسپم ایمیل نرمت کنید 9 تا yes, 5 تا No داریم:

$$Entropy(S) = \sum_{i=1}^{n} -P_i \log_2 P_i = -P_{yes}\log(P_{yes}) - P_{No}\log(P_{No}) = \frac{-9}{14}\cdot\log\left(\frac{9}{14}\right) - \frac{5}{14}\cdot\log\left(\frac{5}{14}\right)$$

$= 0,939$ ← نتیجه: آنتروپی شد 0,939 که عددیه بین 0 (کاملا مثبت) و 1 (کاملا نامثبت) است.

اگر همه داده ها از یک کلاس باشند ← آنتروپی = 0 / اگر نصف yes و نصف No باشه ← آنتروپی = 1

در ساخت درخت تصمیم، در هر مرحله باید بهترین ویژگی رو برای تقسیم (split) داده ها انتخاب کنیم. ویژگی ای خوبه که بیشترین اطلاعات درباره برچسب نهایی (label) به ما بده.

Information Gain = Entropy after split − Entropy before split

$$\text{Entropy after split} = \sum_{i=1} \frac{n_i}{n} \cdot Entropy(D_i)$$

مثال: فرض کن یک مجموعه آموزشی داریم با 10 داده که آیا یک نفر فوتبال بازی میکند یا نه، یک ویژگی ساده داریم به نام هوا که میتواند دو مقدار Sunny یا Rainy داشته باشد.

(I) محاسبه آنتروپی کل (قبل از تقسیم): Yes 5, No 5 ← $-\left(\frac{5}{10}\log\frac{5}{10} + \frac{5}{10}\log\frac{5}{10}\right) = 1$

(II) تقسیم داده ها بر اساس فیچر آب و هوا ← Sunny ← Yes 3, No 2 / Rainy ← Yes 2, No 3

محاسبه آنتروپی آفتابی → $\frac{3}{5}\log\frac{3}{5} + \frac{2}{5}\log\frac{2}{5} \to 0,97$

بارانی ~ ~ → 0,97

(III) ← محاسبه آنتروپی بعد از تقسیم ← $0,97\times\frac{5}{10} + 0,97\times\frac{5}{10} = 0,97$

(IV) محاسبه Gain ← $1 - 0,97 = 0,03$

تعریف و فرمول Gini: Gini یک معیار برای اندازه گیری ناخالصی داده ها در یک گره است هرچه Gini کمتر باشد، گره خالص تر است. (بیشتر متعلق به یک کلاس خاص است).

$$Gini(D) = 1 - \sum_{i=1}^{c} P_i^2 \qquad Gini_{split} = \sum_{i}^{k} \frac{|D_i|}{|D|} \times Gini(D_i)$$

پس الان توی انتخاب فیچر مناسب برای اینکه درخت تصمیم رو بسازیم فیچری خوبه که آنتروپی و Gini آن روی Information Gain به 0 نزدیک تر بشه.

چرا به روش های Ensemble (تجمیعی) نیاز داریم؟ ما همیشه میخوایم مدلی بسازیم که روی داده های جدید (تست) خوب عمل کنه، نه فقط روی داده های آموزش، اما گاهی یه مدل ساده مثل درخت تصمیم عمیق یا، باياس بالایی داره و داده ها رو درست یاد نمیگیره یا یه مدل پیچیده مثل درخت های خیلی عمیق، واریانس بالایی داره و روی داده های جدید عملکرد بدی داره چون زیاد به داده های آموزش چسبیده (overfit شده)

Ensemble methods

راه حل: به جای ساختن یه مدل، چند مدل بساز و جواب هاشونو ترکیب کن این میشه اساس

Bagging ← هدف کاهش واریانس: یعنی جلوگیری از اورفیت با ساختن چند مدل که کمی با هم فرق دارند

روند کار: از داده های آموزش، با نمونه گیری جایگذینی چند زیرمجموعه میسازیم، مثلا" داده اصلی ۱۰۰ تاییه، هر بار یه نمونه ۱۰۰ تایی انجام میدیم ولی ممکنه بعضی داده ها تکرار بشن این کار رو ۱۰ بار مثلا" تکرار میکنیم ← ۱۰ تا مجموعه داده میسازیم روی هر مجموعه یه مدل (مثلا" درخت تصمیم) آموزش میدیم. برای پیشبینی تو کلاسیفیکیشن همه مدل ها رای میدن و Mode میگیریم تو رگرسیون ماشین بیش بی ها رو میگیریم. مزایا: کاهش اورفیت / ساده و موثر / پایه الگوریتم رندوم فارست (گفته بشه)

Boosting ← هدف کاهش باياس: یعنی مدل رو مجبور کنیم بیشتر یاد بگیره و اشتباهشو اصلاح کنه.

روند کار: همه داده ها در ابتدا وزن یکسان دارن. / مدل اول آموزش میبینه / داده هایی که مدل اول اشتباه پیشبینی کرده، وزنشون بیشتر میشه. / مدل دوم یاد میگیره روی اون داده های سخت تر آموزش ببینه / این روند برای چندین مدل تکرار میشه (مثلا" ۱۰۰ تا) / مدل های نهایی با وزن هایی خاص ترکیب میشن.

مزایا: دقت بالا / اصلاح تدریجی اشتباهات / پایه الگوریتم های قدرتمند مثل XGboost.

رندم فارست: همون پایه اش Bagging هست: با استفاده از چندین درخت تصمیم به صورت تصادفی و مستقل ساخته میشن و برای گیری اکثریت یا میانگین گرفتن خروجی نهایی رو میدن.

مدل ML پرسپترون (Perceptron):

یک مدل خطی برای طبقه بندی دو دویی هست، یعنی ورودی رو میگیره و باید تصمیم بگیره که خروجی باید ۱+ باشه یا ۱-. ساختار: ما یک بردار فیچر به اسم $x = [x_1, \dots, x_n]$ و یک بردار وزن $w = [w_1, \dots, w_n]$ و یک مقدار باياس b.

محاسبه فعالسازی (Activation): $w \cdot x + b = \sum_{i=1}^{n} w_i x_i + b$

قانون تصمیم (Decision Rule): if activ $> 0 \rightarrow$ output $= +1$ else output $= -1$

مرحله یادگیری در پرسپترون: خوشحال کن توی ترین (x_i, y_i) که $y = +1$ یا -1 همون لیبل هست.

حالا $\hat{y}$ همون پیشبینی ات ← اگه $\hat{y} = \text{sign}(w \cdot x + b)$ (اشتباه بود) $y \neq \hat{y}$ وزن را آپدیت میکنیم

$\rightarrow w_{new} = x \cdot y + w$, $b_{new} = b + y$

مثال: $x = [2, 3]$, $w = [1, -1]$, $b = 0$, $act = 1 \times 2 + (-1) \times 3 = -1 \rightarrow output = -1$ SOBHAN

پرسپترون چند کلاسه (multiclass Decision Rule) : در دو تایی فقط دو کلاس داشتیم اما
برای چند کلاسه ممکنه ۳ تا یا بیشتر کلاس داشته باشیم مثلا کلاس ۰ = گربه / کلاس ۱ = سگ / کلاس ۲ = اسب
برای هر بردار x ، امتیاز هر کلاس را با بردار وزنش محاسبه میکنیم سپس کلاس
با بیشترین امتیاز انتخاب میشه : $\text{score}_k = w_k \cdot x$ $\quad \hat{y} = \arg\max(w_k \cdot x)$

فرض کن برچسب واقعی y^* با پیش بینی مدل $\hat{y}$ یکیه. اگر $\hat{y} = y^*$ یعنی پیش بینی درست
که در این صورت هیچ تغییری در وزن ها نمیدیم در غیر اینصورت دو کار انجام میدیم.

$w_{y^*} = w_{y^*} + x$
$w_{\hat{y}} = w_{\hat{y}} - x$

مثال: فرض کن ما فقط ۳ کلاس داریم و ویژگی ها ۲ بعدی هستن.

همه وزن ها اول صفر هستن : $w_0 = [0,0]$, $w_1 = [0,0]$, $w_2 = [0,0]$

یک نمونه آموزش : ویژگی ها $x = [1,2]$ کلاس واقعی : $y^* = 1$

حالا مدل با وزن فعلی چیکار میکنه ؟ $\text{score}_i = [1,2] \cdot [0,0] = 0$

فرض کنیم مدل پیش بینی کنه کلاس ۰ یعنی $\hat{y} = 0$ ، چون اشتباه کرده پس :

$w_1 = w_1 + x = [2,1]$, $w_0 = w_0 - x = [-2,-1]$

نتیجه → $w_0 = [-2,-1]$, $w_1 = [2,1]$, $w_2 = [0,0]$

فرمول های Kernelized : $w = \sum_i^N \alpha_{i,y} f(x_i)$

N = تعداد نمونه های آموزشی / α_i : ضریب آپدیت ها برای نمونه i ام / y : برچسب کلاس نمونه i ام / x = ورودی

$$\text{Score}(y,x) = w_y \cdot f(x) = \sum_i \alpha_{i,y} k(x_i, x)$$

$$x \text{ classification of new input} = \text{sign}\left(\sum_{i=1}^{N} \alpha_i y_i k(x_i, x)\right) = \hat{y}$$

$k(x_i, x)$ = کرنل ، تابعی که شباهت بین x_i و x را محاسبه میکند

آپدیت ها → $\alpha_{y,n} = \alpha_{y,n} - 1$ $\quad w_y = w_y - f(x_n)$
$\alpha_{y^*,n} = \alpha_{y^*,n} + 1$ $\quad w_{y^*} = w_{y^*} + f(x_n)$

مثال: فرض کن دو نمونه آموزش داریم

$x_1 = (0,1)$ $\quad x_2 = (1,0)$
$y_1 = +1$ $\quad y_2 = -1$ $\quad k(a,b) = a \cdot b$

همه $\alpha_i = 0$ ، $w_i = 0$ (I)

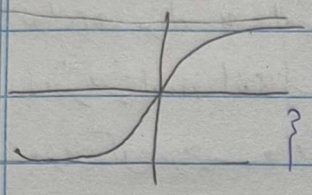
$\hat{y}_1 = \text{sign}\left(\sum_{i=1}^{2} \alpha_i y_i k(x, x_i)\right) = 0 \xrightarrow{\text{آپدیت}} \alpha_1 = 0 + 1 = 1$ (II)

$\hat{y}_2 = \text{sign}(\alpha_1 y_1 k(x_1, x_2) + \alpha_2 y_2 k(x_2, x_2)) = 0 \xrightarrow{\text{آپدیت}} \alpha_2 = 0 + 1 = 1$

$\hat{y} = \text{sign}(\alpha_1 y_1 k(x_1, x) + \alpha_2 y_2 k(x_2, x))$ پیش بینی برای نمونه جدید $x = (1,1)$

$= \text{sign}(1 \times (+1) \times (1 \times 1 + 0 \times 1) + 1 \times (-1) \times (0 \times 1 + 1 \times 1)) = \text{sign}(1 - 1) = \text{sign}(0)$

تابع سیگموید: $\phi(z) = \frac{1}{1+e^{-z}}$ تابعی است که ورودی عددی (z) را گرفته و خروجی را به عددی بین ۰ و ۱ تبدیل میکند. وقتی z بزرگ مثبت است $\to 1$ / بزرگ منفی $\to 0$



$$\text{Best } w \to \max_w LL(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

$$P(y^{(i)} = +1 | x^{(i)}; w) = \frac{1}{1+e^{-w \cdot f(x^{(i)})}}$$

$$P(y^{(i)} = -1 | x^{(i)}; w) = 1 - \frac{1}{1+e^{-w \cdot f(x^{(i)})}}$$

$$P(y^{(i)} | x^{(i)}; w) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}$$

خلاصه کار از طبقه بندی خطی جهت کلاسه توسط خودم که شبیه GPT رفت:
ماتریکسی دیتا داریم و همینطور فرض می کنیم مثلاً ۵ کلاس داشته باشیم توی بخش train ماباری هر دیتا کلاس مربوطه اون رو داریم اول برای هر کلاس به صورت رندوم وزن در نظر میگیریم حالا یه دیتا و تگ اون دیتا را با وزن هر سه کلاس جدا داخل می کنیم و جمع می کنیم در نهایت برای دیتای اول سه تا score از هر کلاس داریم حالا اون کلاسی که بیشترین امتیاز رو داشت مسئله حواب پیشبینی ما، اگه درست بود که هیچی، اگه اشتباه بود وزن کلاس صحیح را تقویت میکنیم و وزن کلاس پیشبینی شده ی اشتباه را تضعیف می کنیم (مثلاً با تا نون پرستون یا کاردان کاهش همینطور، با روش softmax میتونیم احتمال هر کلاس رو هم در نظر بگیریم که از فرمول $\frac{e^{z_i}}{e^{z_1}+e^{z_2}+e^{z_3}}$ به دست میاد حالا برای هر دیتا همین کار را و انجام میدیم تا بخش train تموم شه و آپدیت وزن ها را به دست بیاریم بعد با استفاده از این وزن های به دست آورده شده همین کار ها رو توی بخش تست انجام میدیم و پیشبینی میکنیم نکته این که توی این بخش چون جواب درست رو نمیدونیم فقط پیشبینی میکنیم و آپدیت نمیکنیم.
computer vision (بینایی کامپیوتر) مثل پردازش کردن تصاویر و ویدئو ها را میبینه و تحلیل میکنه مثلاً میگه این عکس سگ هست یا این علیه یا ...
Cnn : نوع خاصی از شبکه عصبی است که برای تحلیل تصاویر طراحی شده است.

NLP (natural language Processing) یعنی آموزش دادن به کامپیوترها که زبان انسان‌ها مثل فارسی، انگلیسی و... رو درک کنن، پردازش کنن و تولید کنن.

word embedding یعنی چی؟ تبدیل هر کلمه به یک بردار عددی (یعنی یک لیست از چند عدد) که ویژگی‌های معنایی اون کلمه رو حفظ میکنه. چون کامپیوتر زبون ما رو نمی‌فهمه و فقط با عدد کار میکنه

برای مثال فرض کنیم بردار embedding اینگونه باشه:

King [0.9, 0.1, 0.8] apple [0.1, 0.7, 0.2]

queen [0.85, 0.1, 0.8] bamnana [0.15, 0.6, 0.25]

بردار King و queen خیلی به هم نزدیکن چون از نظر معنی هم شبیه‌ان

بردار apple و bamaa هم نزدیکن چون هر دو میوه‌ان

ولی بردار King و bannan خیلی فرق دارن.

RNN: شبکه عصبی بازگشتی. یک نوع شبکه عصبی برای پردازش دنباله‌ها هست. در RNN، خروجی زمان فعلی (یا حالت پنهان hidden state) به ورودی زمان بعدی داده میشه

برای هر زمان t:

input[t] + Hidden[t-1] ⇒ Hidden[t] --

Hidden[t] ⇒ output[t]

توی RNN ترتیب مهمه

ایده RNN: حافظه داشته باشیم تا بدونیم قبل از این لحظه چه اتفاقی افتاده اینگونه طراحی شده تا خلاصه‌ای از گذشته رو در خودش نگه داره و باهاش تصمیم بگیره.

مثال: جمله ما: I love NLP که سه کلمه است یعنی ۳ ورودی داریم: x_0 و x_1 و x_2

اندازه ورودی‌ها (Embedding): 2 اندازه حافظه (hidden state): 3

W_x ← وزن‌های ورودی / W_h ← وزن‌های حافظه / تابع فعالسازی: tanh

زمان $t=0$ (کلمه اول: I) ← ورودی اول (embedding) $x_0 = [1, 2]$

حافظه قبلی (اولش صفره) ← $h_{-1} = [0, 0, 0]$

حافظه زمان ۰ ← $h_0 = \tanh(W_x * x_0 + W_h * h_{-1}) = \tanh(\dots)$ "مثلا" $h_0 = [0.5, -0.2, 0.1]$

خروجی ← $y_0 = W_y * h_0$

همینطور برای کلمه‌های دیگه هم میریم.

نکات: ۱ که فقط یه برچسب برای کل جمله میده (مثبت/منفی) خروجی آخر مهمه ولی اگه هر چیزی از جمله یه خروجی بده (مثل ترجمه یا برچسب‌گذاری) اون موقع خروجی‌های وسطا هم مهمن)

وزن‌های W_y و W_x و W_h رو ما یقین نمی‌کنیم خود مدل توی حال آموزش یاد میگیره.

Attention: یعنی مدل به جای اینکه همه اطلاعات قبل رو به یه اندازه نگاه کنه، فقط روی بخش‌های مهمتر تمرکز می‌کنه

فرض کن داریم ترجمه این جمله رو انجام میدیم: The book is on the table.

مدل میخواد ترجمه کنه کلمه‌ی table رو ، به جای اینکه فقط از آخرین حالت (h4) استفاده کنه میاد وزن میده به همه‌ی حالت‌های قبلی (h1 تا h4) و میگه مثلا:

the → 0.1 / book → 0.1 / is → 0.05 / on → 0.15 / the → 0.2 / table → 0.4 (وزن توجه)

فرمول کلی: $Softmax(\frac{Q \cdot K^T}{\sqrt{dk}}) \cdot V = Attention(Q, K, V)$

Query = Q

key = K / value = V / dk = تعداد ستون‌های K / Softmax = تابع نرمال‌سازی

Machine Translation (MT): ترجمه ماشینی یعنی تبدیل یک جمله از یک زبان به زبان دیگه. مدل‌های جدید MT معمولی بر یادگیری عمیق هستن و معمولا از معماری‌های Encoder-Decoder استفاده می‌کنن.

transformer: مدلی پیشرفته برای پردازش دنباله‌ای از کلمات بدون استفاده از recurrence. برخلاف RNN، امکان موازی‌سازی بالا و سرعت بیتر داره. مبنای اصلی GPT و BERT است.

Self-Attention: مکانیزمی که به هر کلمه اجازه می‌ده به سایر کلمات جمله توجه کنه تا معنی دقیق‌تری پیدا کنه. به دست آوردن self-Attention مدل میسنجه هر کلمه چقدر برای درک دقیق مهم است.

Encoder: یعنی از Transformer که ورودی (مثلا جمله زبان اول) را دریافت و معنای آن را در قالب برداری فشرده می‌سازد.

Decoder: یعنی از Transformer که از روی خروجی Encoder و کلمات تولید شده قبلی، جمله بعدی را به زبان مقصد تولید می‌کند.

Propositional logic: همون جبرای منطق که تو کنکور هم بود. هدف: استفاده از قوانین استنتاج برای نتیجه‌گیری. مثال: Rain → wet ، Rain = T پس نتیجه میگیریم wet (زمین خیس است).

Reasoning over Time/Space: استدلال وقتی که محیط با زمان یا مکان تغییر می‌کند.

مثال: در زمان t_0: ربات در اتاق A هست / در زمان t_1: اگر به شرق حرکت کند به B میرود / استنتاج: بعد از یک حرکت شرقی ربات به B میرود.

Knowledge-Based Agent: عاملی که با پایگاه دانش (KB) تصمیم میگیرد، عملیات: TELL (اضافه کردن دانش)، Ask (پرسیدن سوال)، Inference (استنتاج دانش جدید) مثال: اگر باران ببارد چتر لازم است. TELL: امروز باران آمد، Ask: آیا باید چتر بردارم؟ استنتاج: بله

Entailment چیه؟ میگیم یک جمله منطقی α (آلفا) از یک پایگاه دانش KB نتیجه (entailed) میشه اگر در هر موقعیتی که KB درست باشه، اون جمله هم درست باشه: $\alpha \models KB$

یعنی هر وقت KB درست باشه، α هم درسته. ولی برعکسش الزامی نیست: یعنی $KB \rightarrow \alpha$ ✓

ولی $\alpha \rightarrow KB$ لازم نیست درست باشه. (هم ارز لزوما نیست)

SOBHAN

DATE : / / SUB :

CNF → E.g. $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$. or ترکیب اند جیزا کلاز که توش

DNF → E.g. $(A \wedge B) \vee (A \wedge \neg C) \vee (A \wedge \neg D)$. or جیزا کلاز که توش and هست

Horn form → $(A \vee \neg B) \wedge (B \vee \neg C \vee \neg D)$ فرمول CNF که توکلاز ها حداکثر یک لیترال مثبت داشته باشه

valid → True , $A \vee \neg A$ $A \rightarrow A$ $(A \wedge (A \rightarrow B)) \rightarrow B$

قضیه → KB ⊨ a if and only if $(KB \rightarrow a)$ is valid

satisfaiable: یعنی یه حالت داشته باشیم و حداقل یه روش لیبل گذاری داشته باشیم تا T بشه.

unsatisfiable: $A \wedge \neg A$ یعنی هیچ راهی نداشته باشیم تا T کنیم.

قضیه → KB ⊨ a if and only if $(KB \wedge \neg a)$ is unsatisfiable

Forward chaining: از فکت های اولیه شروع میکنه و به کمک قوانین نتیجه های جدید میسازه تا برسه به هدف.

Backward chaining: از هدف شروع میکنه و عقب گرد میکنه و بررسی میکنه که چه fact یا قوانین لازمه تا هدف درست بشه. مثل اینکه یه معمای ریاضی داری و از جواب شروع میکنی تا ببینی چرا

Resolution Rule: $(C \vee A) \rightarrow (C \vee B), (\neg B \vee A)$

Resolution Algorithm: برای اثبات اینکه ① اول $KB \wedge \neg a$ میذاریم : $a \models KB$

② همه جملات رو به CNF تبدیل می کنیم ③ شروع می کنیم از روی جملات، طبق قاعده ی رزولوشن، کلازهای جدید میسازیم. ④ اگر در نهایت به جمله خالی رسیدیم یعنی تناقض پیدا کردیم پس استنتاج درست بود.

مثال: ما داریم: $P \vee Q$ ، $\neg Q \vee R$ ، $\neg P$ میخواهیم R را اثبات کنیم: $\neg R$ را اضافه می کنیم

$KB = \{P \vee Q, \neg Q \vee R, \neg P\} \rightarrow$ ① $\neg Q \vee R$, $\neg R \rightarrow \neg Q$ ② $P \vee Q$, $\neg Q \rightarrow P$ ③ $P, \neg P \rightarrow$ خالی

FOL: First_order logic میگه جهان فقط شامل گزاره های درست/غلط نیست، بلکه شامل ① objects: (اشیاء) مثل انسان، خانه، عدد، رنگ، چک، قانون و ... ② Relation: (رابطه بین اشیا) مثل $bigger_than(x,y) \rightarrow y$ بزرگتر است ، ③ Function: برابر y است $= brother_of(x,y)$ ، $father_of(x) \leftarrow$ پدر x ؛ مثل (توابع)

قدرت FOL : با FOL میتونی جمله کلی بنویسی مثل $\forall x \forall y (Piz(x) \wedge Adjacent(x,y) \rightarrow Breeze(y))$

یعنی برای هر خونه x و y اگر x حفره دارد و y به x مجاور، همه هستند آنگاه y نسیم دارد.

مثال: بعنیت حل آسون یک

1) $Buffalo(x) \wedge Pig(y) \rightarrow Faster(x,y) \rightarrow x = bob, y = Pat$

2) $Pig(y) \wedge Slug(z) \rightarrow faster(y,z) \rightarrow y = Pat, z = Stere$

3) $faster(x,y) \wedge faster(y,z) \rightarrow faster(x,z)$

4) buffalo(Bob)

5) Pig(Pat)

6) slug(steve)

Snook

SOBHAN

Faster(bob, Stere)?

$P(A,B) = P(A|B)P(B) = P(B|A)P(A)$

قانون بیز

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{P(B|A)P(A)}{P(B|+a)P(+a) + P(B|-a)P(-a)}$$

مثال: احتمال باران آمدن ۰٫۰۱۴ میباشد، فرض کنیم هواشناس پیشبینی کرده که فردا باران میبارد میدانیم وقتی باران ببارد هواشناس ۰٫۹ درست میگه و اگر باران نیاد هواشناس ۰٫۱ مگه بارون میاد. حالا با توجه به اینکه هواشناس گفته فردا باران میاد احتمال اینکه واقعاً بیاد؟

$$P(rain|Predict) = \frac{P(Predict|rain)P(rain)}{P(Predict)} = \frac{P(Predict|rain)P(rain)}{P(Predict|rain)P(rain) + P(Predict|\neg rain)P(\neg rain)}$$

$$= \frac{0,9 \times 0,014}{0,9 \times 0,014 + 0,1 \times 0,986} \approx 0,111$$

مثال: اگر داشته باشیم $P(Spam) = 0,2$

$$model: P(Y|w_1, w_2, \dots, w_n) = P(y) \cdot \prod_i P(w_i|Y)$$

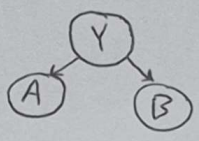
$$P("money"|spam) = 0,8 \quad , \quad P("win"|spam) = 0,6 \quad , \quad P("free"|spam) = 0,7$$

$$P(spam|free, win, money) \propto P(spam) \cdot P("free"|spam) \cdot P("win"|spam) \cdot P("money"|spam) = 0,2 \cdot 0,7 \cdot 0,6 \cdot 0,8 = 0,0672$$

مثال: $(+ \to 0)$ $(- \to 1)$

A	1	1	1	1	0	1	0	1	1	1
B	1	0	0	1	1	1	1	0	1	1
Y	1	1	0	0	0	1	1	0	0	0

مثال Naive Bayes:



الف) جدول توزیع‌های maximum likelihood:

$P(Y)$:

$-y$	$2/5$
$+y$	$3/5$

$P(B|Y)$:

$+b$	$+y$	$1/3$
$-b$	$+y$	$2/3$
$+b$	$-y$	$1/4$
$-b$	$-y$	$3/4$

$P(A|Y)$:

$+a$	$+y$	$1/6$
$-a$	$+y$	$5/6$
$+a$	$-y$	$1/4$
$-a$	$-y$	$3/4$

ب) اگر وردی $A=1$, $B=1$ داشته باشیم خروجی کلسیفایر Y چی هست؟

$$P(+y, -A, -B) = P(+y) \cdot P(-a|+y) \cdot P(-b|+y) = \frac{3}{5} \times \frac{5}{6} \times \frac{2}{3} = \frac{1}{3}$$

$$P(-y, -A, -B) = P(-y) \cdot P(-a|-y) \cdot P(-b|-y) = \frac{2}{5} \times \frac{3}{4} \times \frac{3}{4} = \frac{9}{40}$$

چون $\frac{1}{3} > \frac{9}{40}$ پس خروجی مسئله $+y$ یا همان ۰.

ج) با laplace smoothing جدول $P(A|y)$ و $k=2$ برابر

$$P(A=0|y=0) = \frac{1+2}{6+2\times 2} = \frac{3}{10} \quad \text{(تعداد کلاس‌های متغیر پیش‌بینی)}$$

$$P(A=1|y=0) = \frac{5+2}{6+2\times 2} = \frac{7}{10}$$

$$P(A=0|y=1) = \frac{1+2}{4+2\times 2} = \frac{3}{8}$$

$$P(A=1|y=1) = \frac{3+2}{4+2\times 2} = \frac{5}{8}$$

مدل مارکوف و زنجیره مارکوف

مدل مارکوف: فرض کن ۳ وضعیت داریم (آفتابی S، ابری C، بارانی R) و احتمال تغییر اینجوریه:

$$P = \begin{matrix} \\ S \\ C \\ R \end{matrix} \begin{matrix} \begin{matrix} S & C & R \end{matrix} \\ \begin{bmatrix} 0,7 & 0,2 & 0,1 \\ 0,3 & 0,4 & 0,3 \\ 0,2 & 0,3 & 0,5 \end{bmatrix} \end{matrix}$$

برای مثال میخوام ببینم بعد از ۱ و ۲ و ۳ روز به چه احتمالی هوا ابری هست؟ (فرض اولیه ← هوا آفتابی)

حالت اولیه ← $v_0 = [1,0,0]$

فرمول ← $v_{t+1} = v_t \cdot P$

$$t_1 \to v_1 = v_0 \cdot P = [1,0,0] \cdot \begin{matrix} \begin{matrix} S & C & R \end{matrix} \\ \begin{bmatrix} 0,7 & 0,2 & 0,1 \\ 0,3 & 0,4 & 0,3 \\ 0,2 & 0,3 & 0,5 \end{bmatrix} \end{matrix} = [0,7, 0,2, 0,1] = v_1$$

خفن ترین سوال مارکوف: خلاصه: $\pi = \{A: 0.5, B: 0.5, C: 0\}$, $P = \begin{matrix} \begin{matrix} A & B & C \end{matrix} \\ \begin{bmatrix} 0,2 & 0,6 & 0,2 \\ 0,9 & 0 & 0,1 \\ 0 & 0,3 & 0,7 \end{bmatrix} \end{matrix}$ حالا احتمال هرکدوم تو مرحله بی‌نهایتیک

$$P_\infty(A) = P(A|A) \times P_\infty(A) + P(A|B) \times P_\infty(B) = 0,2 \times P_\infty(A) + 0,9 \times P_\infty(B) \to 0,8 P_\infty(A) = 0,9 P_\infty(B)$$

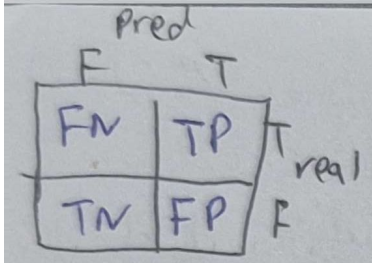
$$P_\infty(B) = P(B|A) \times P_\infty(A) + P(B|C) \times P_\infty(C) = 0,6 P_A(\infty) + 0,3 \times P_\infty(C) \to P_B = 0,6 P_A + 0,3 P_C$$

$$P_\infty(C) = P(C|A) \times P_\infty(A) + P(C|B) \times P_\infty(B) + P(C|C) \times P_\infty(C) = 0,2 P_\infty(A) + 0,1 \times P_\infty(B) + 0,7 \times P_\infty(C)$$

$$\to 0,3 P_C = 0,2 P_A + 0,1 P_B \quad , \quad P_A + P_B + P_C = 1$$

$$P_\infty(C) = 0,338 \quad , \quad P_\infty(A) = 0,350 \quad , \quad P_\infty(B) = 0,312$$

سه معادله سه مجهول ←



Precision ← نسبت پیشبینی های مثبت درست به کل پیشبینی های مثبت ← $\frac{TP}{TP+FP}$

معنی: مدل وقتی میگه مثبت چقدر احتمال داره درست گفته باشه؟

recall ← نسبت پیشبینی های مثبت درست به کل نمونه های واقعی مثبت ← $\frac{TP}{TP+FN}$

معنی: از بین تمام نمونه های واقعی مثبت، چند تاشو مدل شناسایی کرده؟

Accuracy ← نسبت کل پیش بینی های درست به کل نمونه ها: ← $\frac{TP+TN}{TP+TN+FP+FN}$

نتیجه گیری کلمات: ① وقتی Recall مهم هست که مطمئن باریم هیچ مورد مثبتی از دست نره. مثلاً "تشخیص سرطان"

② وقتی Precision مهم است که مطمئن باریم مثبت ها واقعا درست باشن (مثلاً "اسپم فیلتر ایمیل"، یعنی خوبی ایمیل درست رو اسپم نزنی)

③ وقتی Accuracy مهمه که داده ها متوازن هستن و همه خطاها اهمیت مشابه دارند.

روش های Gradient Descent ①: Batch Gradient Descent: کل دیتاست برای هر آپدیت استفاده میشه و خیلی دقیق ولی کند

② Stochastic Gradient Decent (SGD): فقط یک نمونه برای هر آپدیت استفاده میشه و سریع شروع میکنه ولی نویز و نوسان داره

③ mini-batch Gradient Decent: هر بار دسته ای از داده ها رو میبره و از لحاظ سرعت و دقت بین آن دو هست.